

Embedded Message Fabric Architecture

Consolidated Network Architecture Document Set

Generated 2026-08-15

Contents

1. 1. Embedded Message Fabric Architecture
2. 2. Architecture Overview
3. 3. Core Protocol and Routing
4. 4. Application Protocols and Services
5. 5. Logical Links and Transports
6. 6. Link Entity Runtime and Status
7. 7. HDLC Logical Link Profile
8. 8. Classical CAN PDU Adapter
9. 9. Prototype and Validation
10. 10. Rationale, Use Cases, and Risks

Embedded Message Fabric Architecture

Status: Consolidated document index and authority guide

Maturity: Draft; suitable for prototype alignment, not a wire-interoperability claim

Scope: Static, bounded embedded messaging for local/private deployments

This directory is being consolidated from exploratory drafts into a smaller document set. The architecture is a statically configured embedded message fabric, not a general internetwork. It supports bounded service messages across local IPC/ICC, MCU links, buses, and explicitly configured gateways.

Reading order

1. [Architecture overview](#) — scope, layers, supported communication domains, invariants, security posture, and maturity.
2. [Core protocol and routing](#) — authoritative endpoint identity, service roles, addressing, Virtual Circuits (VCs), listeners, forwarding, and replication.
3. [Application protocols and services](#) — authoritative service archetypes, endpoint bindings, pub/sub behavior, schemas, sizing, and local endpoint behavior.
4. [Logical links and transports](#) — authoritative link contract, transport semantics, concurrency, and link integrations.
5. [Link Entity runtime and status](#) — authoritative runtime-link ownership, lifecycle, recovery, supervision, and status responsibilities.
6. [HDLC logical-link profile](#) — experimental serial/FIFO framing candidate and stabilization gaps.
7. [CAN PDU adapter specification](#) — experimental Classical CAN profile and its open wire-format issues.
8. [Prototype and validation](#) — implementation boundary, test matrix, conformance work, and promotion criteria.
9. [Rationale, use cases, and risks](#) — non-normative motivation, deployment fit, tradeoffs, alternatives, adoption strategy, and risks.

Document status and ownership

Document	Status	Owns
<code>README.md</code>	Index / authority guide	Reading order, document authority, maturity summary, and source traceability
<code>architecture_overview.md</code>	Consolidated draft	Architectural scope, boundaries, high-level model, and invariants
<code>core_protocol_and_routing.md</code>	Provisional core specification	Canonical protocol and routing definitions
<code>application_protocols_and_services.md</code>	Consolidated provisional design contract	Application/service contracts and archetypes
<code>logical_links_and_transports.md</code>	Stable generic contracts; wire profiles unstabilized	Logical-link and transport contracts

Document	Status	Owns
<code>link_entity_runtime_and_status.md</code>	Provisional runtime contract	Runtime-link ownership, lifecycle, recovery, supervision, and status
<code>hdlc_logical_link_profile.md</code>	Experimental / not a stable wire profile	Serial/FIFO framing candidate
<code>can_pdu_adapter_spec.md</code>	Experimental / work in progress	Classical CAN mapping and PDU adaptation
<code>prototype_and_validation.md</code>	Actionable prototype plan; disposable validation implementation.	Prototype scope, tests, vectors, and readiness gates
<code>rationale_use_cases_and_risks.md</code>	Non-normative consolidated draft	Motivation, examples, alternatives, adoption, and risk analysis

Current maturity

- **Architecture direction — consolidated draft.** The system is a static, bounded embedded message fabric. Service upstream/downstream roles are independent of physical topology.
- **Communication domains — supported direction.** `VirtualCircuit::kLocalHost` supports same-host IPC/ICC; `VirtualCircuit::kMainCompute` supports Main-to-Main communication; configured VCs support Main/ECU and multi-link deployments.
- **Routing scope — constrained.** A downstream target is the complete `VCN + Index`. `UpstreamHost` identifies a configured Main for external/Main relationships and reserves symbolic `UpstreamHost::kLocalHost` for local IPC/ICC. It is distinct from `VirtualCircuit::kLocalHost`; both numeric encodings remain provisional. Arbitrary ECU-to-ECU unicast, especially across buses, is not supported.
- **Publication — supported direction.** A configured listener on an attached link may consume a visible publication; listening itself creates no forwarding or fan-out. Transparent forwarding may make a publication visible on another attached link within the same VC. Explicit configured cross-VC state replication remains separate.
- **Unreliable Datagram — completed current transport definition.** Exact interoperable wire profiles still depend on their owning link specifications and conformance vectors.
- **Sequenced/E2E-Protected Datagram — Provisional.** Required semantics and exact profile remain to be completed.
- **Reliable delivery — wholly TBD.** No reliable wire format, state machine, or interoperability is claimed.
- **CAN and HDLC — experimental.** Encodings, integrity details, error behavior, and conformance vectors are not yet stable interoperability contracts.
- **Security — initial local/private posture.** The initial prototype excludes shell, memory/MMIO access, update, fault control, replay, and raw injection. Experimental versions require a separate development build and cannot be enabled solely by runtime network configuration. IP composition is possible, but a secure remote gateway is not an initial feature.

Terminology

Use [core protocol and routing](#) for normative definitions of `VCN`, `Index`, `UpstreamHost`, service direction, endpoint identity, listener, forwarder, and replicator. Use [application protocols and services](#) for service and pub/sub terminology, [logical links and transports](#) for transport and link terminology, and [Link Entity runtime and status](#) for runtime ownership, recovery, and status.

This index and the [overview](#) use those terms only to explain scope. They do not define numeric values, bit layouts, serialization, or wire compatibility.

Known gaps

- Final canonical numeric encodings and immutable named wire profiles.
- Golden/conformance vectors and independent interoperability testing.
- Complete Sequenced/E2E-Protected Datagram metadata, validation, reset, freshness, and failure semantics.
- Complete Reliable transport peer model, state machine, flow control, retransmission, reset, and resource bounds.
- Stable CAN and HDLC framing, integrity coverage, malformed-input behavior, and profile selection.
- Generated VC/route configuration, loop and reachability validation, capacity analysis, and deployed-configuration fingerprints.
- Final endpoint/router/link APIs, bounded backpressure behavior, and local IPC memory/coherency requirements.
- A reviewed security architecture for authentication, authorization, replay protection, key lifecycle, audit, and remote gateways.
- Production criteria for safety-relevant, vehicle, industrial, or space deployments.

Open gaps remain explicit. A draft's proposed number, layout, or algorithm does not become authoritative merely because the corresponding consolidated specification is incomplete.

Authority: consolidated set versus source drafts

The consolidated document set is the current authority within each document's declared scope. Topic-specific normative documents own detailed definitions; this README owns document authority; the overview owns only architecture-level boundaries and invariants; the rationale document is non-normative.

When a consolidated document conflicts with a source draft, the consolidated document controls. When a delegated consolidated document marks an issue unresolved, the issue remains unresolved; an older draft does not regain normative authority. The controlling decisions recorded in the consolidation effort override contrary draft material.

All original Markdown drafts remain unchanged as historical inputs for provenance, design intent, and recovery of omitted context. They are not deleted, rewritten, or silently promoted into the current specification.

Consolidated specifications remain in this directory; historical inputs are stored in `../drafts/network/`.

Source traceability

Classification means:

- **Baseline** — starting point for consolidation, subject to later controlling decisions.
- **Specialized input** — topic-focused material retained, reconciled, or explicitly deferred in an owning consolidated document.
- **Superseded input** — an older lineage consulted for provenance but not current authority.
- **Historical input** — early intent or context retained without normative force.

Source draft	Classification	Primary consolidation use
<code>network_stack_design_updated.md</code>	Baseline	Whole architecture; reconciled into all owning documents
<code>network_stack.md</code>	Superseded input	Early stack model and terminology history
<code>network_stack_ecosystem_updated.md</code>	Superseded input	Ecosystem, layering, and service portability lineage
<code>recent_protocol_design_updates.md</code>	Specialized input	Service roles, congestion, scheduling, diagnostics, and bounded behavior
<code>network_architecture_refinements.md</code>	Specialized input	VC topology, local/Main domains, validation, security, and MVP direction
<code>static_virtual_circuit_routing.md</code>	Specialized input	Static VC concepts and configured forwarding
<code>link_entity_recovery_and_status.md</code>	Specialized input	Link/entity lifecycle, recovery, and status
<code>endpoint_identifiers_revised.md</code>	Specialized input	Endpoint-identity lineage; reconciled with later controlling decisions
<code>endpoint_identifiers.md</code>	Superseded input	Earlier endpoint-identity model
<code>application_protocol_archetypes_updated.md</code>	Specialized input	Prototype service archetypes and IPC/ICC cases
<code>application_protocol_archetypes.md</code>	Superseded input	Earlier archetype lineage
<code>service_catalog_and_schema_conventions.md</code>	Specialized input	Service catalog, schema, compatibility, and naming guidance
<code>service_size.md</code>	Specialized input	Message-size and constrained-link analysis
<code>flexible_logical_link_layer_design.md</code>	Specialized input	Generic logical-link responsibilities and APIs
<code>transport.md</code>	Specialized input	Transport design history and unresolved reliability work
<code>pooled_linked_spvc_design.md</code>	Specialized input	Bounded local queue/concurrency mechanism

Source draft	Classification	Primary consolidation use
<code>can_pdu_adapter_reorganized.md</code>	Specialized input	Latest experimental CAN PDU adapter lineage
<code>can_pdu_adapter.md</code>	Superseded input	Earlier CAN adapter design
<code>can_pdu_payload_capacity.md</code>	Specialized input	Canonical specialized CAN capacity analysis
<code>can_pdu_payload_capacity(1).md</code>	Superseded input	Exact duplicate of the canonical CAN capacity input
<code>hdlc_service_protocol_design_rev3.md</code>	Specialized input	Latest experimental HDLC design lineage
<code>hdlc_service_protocol_design.md</code>	Superseded input	Earlier HDLC design
<code>distributed_embedded_system_examples.md</code>	Specialized input	Board, HIL, robot, vehicle, and satellite examples
<code>project_adoption_and_security_risks.md</code>	Specialized input	Adoption strategy and security guardrails
<code>RFP.md</code>	Specialized input	Reliable Transport design requirements and history

These 25 drafts form the traceability baseline. Their retention does not imply equal maturity or authority.

Architecture Overview

Status: Consolidated architecture draft

Authority: Normative for architectural scope, boundaries, and invariants; detailed definitions are delegated

Interoperability: Not claimed; unresolved numeric encodings and wire profiles remain open

Purpose

This architecture defines a small, statically configured message fabric for embedded systems. It carries discrete, bounded service messages across local software, multicore devices, embedded buses, point-to-point links, and explicitly configured gateways.

It is intended to let service protocols remain stable while deployment placement and compatible link technology change. It is not a smaller version of the Internet and does not make arbitrary nodes globally addressable.

Goals

- Stable boundaries between application services, endpoint dispatch, transport semantics, logical links, and hardware links.
- Bounded behavior suitable for MCUs, FPGA softcores, RTOS systems, and host tools.
- One service model for same-core calls, inter-core IPC/ICC, Main/ECU traffic, Main-to-Main traffic, and constrained static forwarding.
- Brokerless publication with known producers and configured consumers.
- Link independence within explicit message-size, timing, and transport constraints.
- Static analysis of routes, capacity, loops, identities, and configuration compatibility.
- Observable congestion, loss, restart, and configuration failures.
- Composition with IP or specialized bulk/streaming systems at explicit boundaries.

Non-goals

- General internetworking, arbitrary meshes, or dynamic routing.
- Arbitrary ECU-to-ECU unicast, especially across physical buses or VCs.
- Runtime route discovery, address assignment, or mandatory service discovery.
- A general broker or globally dynamic subscription system.
- A universal reliable byte stream or transparent segmentation of unbounded application data.
- Making every service support every transport or link.
- Implicit remote access, raw link injection, or a built-in secure Internet gateway.
- Claiming safety certification, cryptographic protection, or wire interoperability from the architecture alone.

Five-layer model

1. **Application / Service** — owns message meaning, roles, timing, freshness, authority, and application failure behavior. See [application protocols and services](#).
2. **Endpoint / Service Router** — binds configured endpoint identities to local producers, consumers, storage, and dispatch behavior. Routing details are owned by [core protocol and routing](#).
3. **Protocol / Transport** — supplies a small set of bounded delivery semantics selected for an endpoint. See [logical links and transports](#).
4. **Logical Link Layer (LLL)** — carries bounded PDUs over a concrete link while preserving or reconstructing the required canonical semantics. Link-specific encoding may differ.
5. **Hardware Link** — provides the fundamental frame, datagram, byte, FIFO, or shared-memory transfer mechanism.

The layers are contracts, not mandatory threads, processes, or allocation boundaries. A constrained implementation may combine them while preserving their responsibilities.

Supported topology and communication domains

The model supports:

- one process or core;
- multiple tasks or cores on one host;
- point-to-point Main/ECU links;
- one or more Main devices and multiple ECUs on a broadcast bus;
- multiple Main devices communicating in the configured `kMainCompute` domain;
- loop-free, statically configured paths across unlike links;
- publications consumed by configured listeners on attached links;
- explicit, configured cross-VC state replication.

Three domain classes are central:

- `VirtualCircuit::kLocalHost` — local IPC/ICC. Traffic remains inside the host and is not emitted on an external link.
- `VirtualCircuit::kMainCompute` — a configured domain for Main-to-Main communication using the same service model as other traffic.
- **Deployment VCs** — statically configured communication domains for Main/ECU and multi-link connectivity.

A system may also compose the fabric with an outer IP network, native stream, file-transfer service, or specialized real-time network. That outer system supplies capabilities outside this architecture and creates an explicit gateway, proxy, or trust boundary.

Service roles and peer selection

Every directed service protocol has an **upstream** role and a **downstream** role. These are service semantics:

- upstream is the stimulus, controller, or initiator side;
- downstream is the responder or controlled side.

The roles do not describe physical hierarchy. They do not reverse when the downstream side sends a response or later asynchronous traffic. A Main, ECU, task, or core can implement either role for a particular service. Symmetric interaction is composed from directed service instances rather than inferred from topology.

Addressing remains deliberately asymmetric and bounded:

- a downstream target is identified by the complete `VCN + Index`; an `Index` is not a global node address;
- `UpstreamHost` identifies a configured Main for external/Main relationships, but reserves symbolic `UpstreamHost::kLocalHost` for local IPC/ICC;
- `VirtualCircuit::kMainCompute` permits Main-to-Main services without redefining service roles;
- `VirtualCircuit::kLocalHost` permits same-core and cross-core services without exposing internal placement externally.

`UpstreamHost::kLocalHost` and `VirtualCircuit::kLocalHost` are distinct; both numeric encodings remain provisional. The canonical fields, endpoint identity, reply binding, and receive/transmit-only cases belong to [core protocol and routing](#). This overview assigns no wire layouts.

Virtual Circuits

A VC is a statically configured connectivity context. It can describe one physical bus, one point-to-point link, or a loop-free path through several links. The message identifies the VC and relevant member/target; deployment configuration contains the physical route.

VCs provide bounded communication and policy domains. They do not create global addresses or authorize arbitrary traffic between domains. Physical loops may exist, but each active VC forwarding topology must be deterministic and loop-free unless a future specification explicitly defines another bounded mechanism.

Exact VC identifiers, target fields, group behavior, ingress rules, and forwarding state are delegated to [core protocol and routing](#).

Publication, listening, forwarding, and replication

These operations are distinct:

- **Publisher** — originates a service value or event under explicit producer authority.
- **Subscriber/consumer** — is a configured recipient that applies the service's storage, timing, and validation behavior.
- **Configured listener** — consumes a publication visible on an attached link without becoming its addressed peer, reliable peer, producer, or forwarding authority.
- **Transparent forwarder/repeater** — carries a received message along a statically configured path within its communication context while preserving the canonical service and transport semantics. It may make a publication visible on another attached link within the same VC, but does not originate application state.
- **Cross-VC replicator** — explicitly consumes selected state in one VC and republishes it into another configured VC. It is a policy and failure-semantics boundary, not transparent forwarding and not general

ECU routing.

Listening itself creates no forwarding or fan-out. Identity preservation or translation at a replicator must be specified by its owning service/configuration. Receivers do not forward or replicate by default. See [core protocol and routing](#) and [application protocols and services](#) for the detailed contracts.

Security posture

The initial supported deployment is local/private and assumes the embedded fabric is physically or host isolated unless a connected device or gateway is compromised. The core architecture does not provide authentication, authorization, confidentiality, or cryptographic replay protection.

Initial implementations and examples must:

- exclude shell, memory/MMIO access, update, fault control, replay, and raw injection from the initial prototype;
- require a separate development build for experimental versions of excluded capabilities; runtime network configuration alone cannot enable them;
- favor read-only observation and diagnostics;
- require explicit configuration for forwarding and cross-VC replication;
- keep remote/IP exposure outside the recommended initial feature set;
- treat SWD/JTAG, memory access, calibration, firmware update, and raw packet injection as privileged trust boundaries.

IP composition remains possible through an outer carrier or explicit proxy. A secure policy-enforcing remote gateway requires a separate reviewed threat model, authentication, authorization, replay protection, key lifecycle, rate limiting, and audit design; it is not an initial architecture feature.

Architecture invariants

1. Messages and all associated queues, retries, and state are bounded by configuration.
2. Application services are link-independent only within declared size, timing, and transport compatibility.
3. Service upstream/downstream roles are independent of physical topology and device class.
4. A downstream target is the full `VCN + Index`; an `Index` has no global meaning.
5. `UpstreamHost` identifies a configured Main for external/Main relationships; reserved symbolic `UpstreamHost::kLocalHost` identifies local IPC/ICC and is not a general host or ECU address.
6. `UpstreamHost::kLocalHost` is distinct from `VirtualCircuit::kLocalHost`; both numeric encodings remain provisional. `VirtualCircuit::kLocalHost` traffic remains local, and `VirtualCircuit::kMainCompute` provides the supported Main-to-Main domain.
7. Arbitrary ECU-to-ECU unicast, particularly across buses or VCs, is unsupported.
8. A configured listener on an attached link may consume a visible publication without becoming a peer or producer; listening creates no forwarding or fan-out.
9. Cross-VC state sharing requires an explicit configured replicator.
10. Forwarding is explicit, static, loop-free by configuration, and non-originating; it may make a publication visible on another attached link within the same VC.

11. Published state has explicit producer/authority rules; replication does not create accidental competing producers.
12. Transport is selected by service semantics. Reliability is not assumed to be safer than loss with freshness detection.
13. Congestion, rejection, drops, stale data, restart, and configuration mismatch require bounded, observable handling.
14. Security-sensitive capabilities are least-privilege and disabled by default.
15. No unresolved number, bit layout, serialization, or profile is implied to be stable.

Current maturity

Area	Status
Scope, layers, role independence, and static VC direction	Consolidated draft
<code>VirtualCircuit::kLocalHost</code> , <code>VirtualCircuit::kMainCompute</code> , Main/ECU domains, listeners, and configured replication	Supported architecture direction
Unreliable Datagram semantics	Completed current transport definition
Sequenced/E2E-Protected Datagram	Provisional
Reliable transport	Wholly TBD
CAN PDU adapter	Experimental
HDLC profile	Experimental
Numeric encodings, immutable wire profiles, and conformance vectors	Unresolved
Secure remote gateway	Not an initial feature

Implementation and promotion criteria belong to [prototype and validation](#). Until the delegated specifications and vectors are complete, independently developed wire implementations must not be presumed interoperable.

Related documents

- [Document index and source traceability](#)
- [Core protocol and routing](#)
- [Application protocols and services](#)
- [Logical links and transports](#)
- [Link Entity runtime and status](#)
- [HDLC logical-link profile](#)
- [CAN PDU adapter specification](#)
- [Prototype and validation](#)
- [Rationale, use cases, and risks](#)

Core Protocol and Routing

1. Status and scope

Status: Provisional core specification. This document is normative for the initial prototype where it uses **SHALL**, **SHALL NOT**, **SHOULD**, or **MAY**, but it is not a frozen wire-protocol release. Items labelled **Provisional** may change after prototype validation. Items labelled **Deferred** are not part of the initial prototype.

This document defines the canonical message descriptor, endpoint identity, service-role direction, Virtual Circuit routing, peer binding, publication, transparent forwarding, static validation, and core error behavior. It defines logical semantics above the Logical Link Layer (LLL); it does not require every LLL to serialize the canonical descriptor byte-for-byte.

The following companion documents are planned as part of the consolidated set:

- [Document map](#) — authority, terminology, document status, and consolidated open-issue index.
- [Architecture overview](#) — goals, layering, deployments, security boundary, and system-wide invariants.
- [Application protocols and services](#) — endpoint APIs, service patterns, schemas, and application error protocols.
- [Logical links and transports](#) — LLL contracts, capabilities, transport profiles, and concurrency.
- [Link Entity runtime and status](#) — Link Entity lifecycle, recovery, observability, and status behavior.
- [Experimental CAN PDU adapter](#) — CAN encoding, framing, reassembly, and integrity details.
- [Prototype and validation](#) — implementation boundary, test matrix, conformance evidence, and promotion criteria.
- [Rationale, use cases, and risks](#) — non-normative examples, alternatives, adoption concerns, and security risks.

Reliable transport state machines and exact CAN behavior are outside this document. The initial prototype uses Unreliable Datagram and must not infer reliable or CAN semantics from this specification.

2. Core terms

- **Main**: a device eligible for a compact `UpstreamHost` identity. Main is a deployment role, not a required service role.
- **ECU**: a non-Main embedded node. This name does not imply a particular processor, operating system, or physical link.
- **Virtual Circuit (VC)**: a statically configured connectivity and policy domain. A **Virtual Circuit Number (VCN)** identifies that domain.
- **Index**: a compact, VCN-scoped member identifier. The same numeric Index MAY identify different members in different VCs.
- **Upstream service role**: the initiator, controller, or stimulus side of one directed service instance.
- **Downstream service role**: the responder or controlled side of that service instance.
- **Producer**: the endpoint instance that originates a publication.

- **ProducerKey**: opaque deployment-local metadata identifying one statically configured publication producer.
- **Passive listener**: a configured consumer that is not the addressed peer and does not participate in peer or transport state.
- **Transparent forwarder**: a configured same-VC relay that preserves canonical end-to-end semantics.
- **Cross-VC replicator**: a configured component that terminates an observation on one VC and originates a distinct authorized publication on another VC.

3. Canonical working descriptor

3.1 Representation

The canonical representation above the LLL is:

Control	8 bits
Routing	16 bits
EndpointId	16 bits

Total	40 bits

This 5-byte logical descriptor is **Provisional**. An LLL MAY encode fields in native metadata, omit values that static configuration reconstructs unambiguously, or use a named compact profile. Before delivery to the endpoint router, it SHALL reconstruct the same canonical values. Wire order, bit order, and profile-specific encodings belong in the LLL specifications.

The working logical partition is:

Control (8)	
Namespace	2
Reserved	2
Multicast control	1
Transport selector	3
Routing (16)	
QoS selector	2
Direction	1
UpstreamHost	3
VCN	5
Index	5
EndpointId (16)	

The widths and partition above are the prototype contract, but the packed bit positions and numeric assignments are not frozen. Reserved control bits and the multicast-control bit SHALL be transmitted as zero in

the initial prototype. A receiver SHALL treat a nonzero unassigned bit as an unsupported or malformed descriptor rather than guessing its meaning.

3.2 Namespace and endpoint identity

Namespace is a 2-bit component of endpoint identity. The provisional allocation is:

0, 1, 2	deployment-defined namespaces
3	open ecosystem namespace; allocation governance remains TBD

The logical endpoint identity is:

(Namespace, EndpointId)

Routing fields, service role, physical location, and transport selection are not part of endpoint identity. A reusable service MAY provide a default identity, but the deployment owns its final allocations and instances.

Literal **EndpointId** values are `0x0100` through `0xFFFF`. Application code, transport code, endpoint routing, capture, and forwarding SHALL use literal IDs.

Values `0x0000` through `0x00FF` are reserved for optional link-local aliases. An alias:

- exists only inside one statically configured LLL;
- SHALL map to exactly one literal ID in that link configuration;
- SHALL be expanded before delivery above the LLL;
- SHALL NOT be exposed to services, transports, routers, or other LLLs;
- does not change endpoint identity.

Aliasing is disabled unless explicitly configured. An alias-bearing profile SHALL fail closed unless both peers derive their alias tables from the same generated topology artifact or have compatible alias digests provisioned by an out-of-band mechanism. An unknown or mismatched digest SHALL disable the alias profile for the affected peer or link, and traffic using that profile SHALL remain disabled. A separately configured literal-ID profile MAY be used instead, but implementations SHALL NOT fall back to or negotiate that profile automatically. Alias-table and digest consistency are configuration-validation requirements.

3.3 QoS

Deferred: numeric QoS ordering is unresolved. The 2-bit field is an opaque selector for a named, profile-defined symbolic class. APIs, configuration, and design documents SHALL use symbolic class names and SHALL NOT assume that a larger or smaller numeric value has higher priority. Class names, numeric mapping, arbitration mapping, and scheduling behavior belong in the link and prototype specifications.

3.4 Direction

Direction selects the **destination service role**. It does not describe physical motion, hierarchy, ingress/egress, or which device is currently transmitting.

Conceptually its two values are:

```
ToUpstreamRole
ToDownstreamRole
```

The exact symbolic and numeric encoding is **Provisional**.

An endpoint's upstream/downstream role is fixed for one directed service instance. A downstream endpoint MAY respond immediately or transmit asynchronously later; when sending to its upstream peer it selects the upstream destination role. This transmission does not reverse either endpoint's role. Genuinely symmetric protocols SHOULD compose two oppositely directed service instances.

3.5 UpstreamHost

`UpstreamHost` normally identifies a configured Main device. It:

- SHALL NOT be interpreted as a general node or ECU address;
- SHALL NOT substitute for the complete downstream target `(VCN, Index)`;
- does not prove that any service role is placed on that Main;
- does not imply physical upstream/downstream topology;
- SHALL be validated against the deployment's configured Main identities.

Local IPC/ICC uses the reserved symbolic value `UpstreamHost::kLocalHost`. This value is not a Main identity or node address and is valid only when the VCN field contains `VirtualCircuit::kLocalHost`. The two symbols belong to different field types; their numeric values are **Provisional** and are not required to match. A router or LLL SHALL reject `UpstreamHost::kLocalHost` on external ingress or egress, and SHALL reject either local symbol without the other.

A Main MAY host either service role. The exact `UpstreamHost` value allocation and the relationship between `UpstreamHost` and a Main's Index on `kMainCompute` are open issues.

4. Virtual Circuits and addressing

4.1 VC semantics

A VCN identifies a static connectivity and policy context, not a physical bus and not a hop-by-hop route. A VC MAY be implemented by one link, several heterogeneous links joined by transparent forwarders, a local IPC mechanism, or a statically selected subset of a physically richer topology.

The packet carries the VCN and Index; configuration contains the physical path. Nodes SHALL NOT discover or install routes dynamically merely because traffic is observed.

An Index is meaningful only within its VCN. Individual Index assignments used as unicast members SHALL be unique within that VC. No globally unique node address is defined by this protocol.

4.2 Downstream target

The complete node-level target for a message addressed to a downstream service role is:

```
(VCN, Index)
```

`EndpointId` and `Namespace` then select the service endpoint at that target. Neither `Index` alone nor `UpstreamHost` alone is a complete downstream target.

This document intentionally does not freeze more general source/destination field behavior. In particular, it does not invent a universal source address field or redefine `UpstreamHost` to fill that gap. Profiles and endpoint APIs shall be validated against the accepted downstream-target rule before broader field behavior is standardized.

4.3 Symbolic reserved VCs

The VCN space reserves the following symbolic identities. Their numeric values are **Provisional** and SHALL be assigned by the consolidated profile:

- `kLocalHost`: communication entirely within one host or node.
- `kMainCompute`: communication among configured Main devices.

Traffic on `VirtualCircuit::kLocalHost` SHALL remain inside the current host and SHALL carry `UpstreamHost::kLocalHost`. It SHALL NOT be emitted onto an external LLL, forwarded by a gateway, or reconstructed from external ingress. A local implementation MAY use `Index` to select local cores, tasks, processes, or routing domains, subject to static configuration. The numeric assignments of both local symbols remain **Provisional**.

`kMainCompute` is an ordinary statically routed VC with Main-only membership. Each participating Main has a VCN-scoped `Index`. Main-to-Main services use the same endpoint identity, `Direction`, peer binding, and forwarding rules as other services. The protocol does not define a separate inter-Main message format.

5. Endpoint peer binding and replies

Peer binding is an endpoint/router configuration behavior, not a new wire field. One endpoint registration SHALL have exactly one of these five binding modes:

1. **Static binding** — a peer or target is fixed by generated configuration.
2. **Learned-from-ingress binding** — a bounded peer binding may be learned only from an authorized ingress mode described below.
3. **Request-scoped binding** — an incoming request yields an opaque `ReplyContext` valid only for the associated operation or bounded lifetime.
4. **Receive-only** — the endpoint consumes configured traffic and has no network transmit peer.
5. **Transmit-only** — the endpoint originates configured traffic and has no inbound peer binding.

Learned-from-ingress binding SHALL be disabled on unauthenticated multi-access LLLs. It MAY be enabled only for either a statically single-peer ingress or a named authenticated-origin profile, and only within a preconfigured allowlist of permitted origin, service, VC, `Index`, and interface constraints. Route, VC, `Index`, interface, and descriptor validation are necessary routing checks; they are not authentication. Each enabled learned mode SHALL define finite capacity, expiry, replacement/eviction policy, restart invalidation or restoration behavior, and auditable learn, replace, expire, reject, and clear events. Prototype conformance SHALL test all enabled binding modes rather than assuming learned-from-ingress binding is unconditionally available.

A protocol that needs request-scoped replies and autonomous transmission SHALL use separate endpoint registrations, or explicit bounded named sub-bindings whose independent authority and lifetime are part of the service configuration. Separate registrations are RECOMMENDED for the initial prototype. Each named sub-binding SHALL have one declared binding mode, authority, and lifetime. This requirement specifies composition, not API names or spelling. A complex multi-peer/session service may manage an explicitly bounded collection of named sub-bindings, but it does not gain arbitrary node addressing.

`ReplyContext` SHALL contain or reference enough validated router state to construct a role-correct reply without requiring application code to manipulate raw VCN, Index, Direction, or `UpstreamHost` fields. It SHALL be opaque to ordinary service code, bounded in lifetime and storage, invalidated according to documented restart/configuration rules, and unusable as authority for a different service operation. Its exact fields are intentionally not specified here.

6. Supported communication patterns

6.1 Main and ECU

A common command pattern places the upstream service role on a Main and the downstream role on an ECU. Traffic to the ECU selects the downstream role and uses the ECU's complete `(VCN, Index)` target. A response or later asynchronous transmission from the ECU selects the upstream destination role and retains the same endpoint roles. `UpstreamHost` identifies the configured Main involved; it does not address the ECU. Local-only service instances instead use the paired `UpstreamHost::kLocalHost` and `VirtualCircuit::kLocalHost` symbols.

The placement may be reversed for a service. For example, an ECU error source may hold the upstream role while a Main logger holds the downstream role. Device category never overrides the service's configured roles.

6.2 Main and Main

Main-to-Main communication occurs on `kMainCompute`. One Main hosts the upstream role and another the downstream role for a directed service instance. The downstream Main is targeted by `(kMainCompute, Index)`. A duplex relationship is two directed instances.

`UpstreamHost` remains a compact Main identity but is not assumed to equal, alias, or derive from the `kMainCompute` Index. That relationship is TBD and configuration SHALL treat the two identities as distinct until resolved.

6.3 ECU observation without ECU unicast

Arbitrary ECU-to-ECU unicast is not supported. In particular, the core router SHALL NOT turn one ECU's Index into a general address for commanding another ECU on the same or a different VC.

An ECU MAY nevertheless consume another ECU's publication when the receiving link exposes the traffic and a passive-listener subscription explicitly allows it. This is observation of a configured producer, not peer addressing.

7. Publications, listeners, and producer identity

A publication SHALL carry runtime metadata containing an opaque, deployment-local `ProducerKey`.

`ProducerKey` is assigned by static configuration for prototype, configuration, and capture use. For received traffic it SHALL be derived or attached only after ingress and routing validation; a locally originating endpoint obtains only the key assigned to its static registration. It is not a frozen wire field. Endpoint identity, physical visibility, and unvalidated source fields SHALL NOT create a `ProducerKey`.

Exactly one producer SHALL be authorized for each `ProducerKey` unless the service defines explicit multi-producer authority and arbitration. Endpoint identity alone may be reused by several nodes, so consumers and capture SHALL retain the validated `ProducerKey`. The exact source-field and return-path semantics remain open and are not frozen by this metadata rule.

A passive listener:

- MAY be one of any number of statically configured listeners consuming an explicitly authorized publication visible on an attached ingress link;
- is not the addressed peer and receives no delivery guarantee;
- SHALL NOT acknowledge, alter flow control, or affect peer state;
- SHALL NOT cause the producer to infer successful delivery;
- SHALL retain the publication's validated `ProducerKey`;
- SHALL preserve producer attribution when storing or explicitly republishing data.

Physical visibility and endpoint identity do not create a subscription, reply authority, or forwarding authority. Listeners and producers SHALL be declared by static configuration. A transparent forwarder MAY carry an authorized publication on the same VC to another attached link; listening alone SHALL NOT cause forwarding. A transparent forwarder or listener never becomes the producer merely because it retransmits or stores a value.

8. Forwarding and cross-VC publication

8.1 Same-VC transparent forwarding

Forwarding is disabled unless explicitly configured. A transparent forwarder MAY continue one VC over another physical link. It SHALL:

- preserve the VCN and Index;
- preserve `(Namespace, EndpointId)`;
- preserve Direction and `UpstreamHost`;
- preserve end-to-end transport state and application payload;
- avoid terminating, acknowledging, regenerating, or translating end-to-end transport state;
- never become the producer or service peer;
- modify only hop-local LLL framing required by the egress profile.

If an egress LLL cannot preserve or unambiguously reconstruct these canonical semantics, the route is invalid. Transparent forwarding never changes VCN.

8.2 Cross-VC replication

Cross-VC publication is not routing. A component that consumes a publication on one VC and emits it on another is a **cross-VC replicator**. It terminates one observation and originates a distinct publication under the destination VC's policy.

Every cross-VC replication edge SHALL be explicitly configured, bounded, observable, loop-free, and distinguishable from a transparent forwarder. Each edge SHALL declare:

- the eligible publication message types;
- the authorized source `ProducerKey`;
- the destination `ProducerKey` and its authority owner;
- how original provenance is represented at the destination; and
- the conflict policy.

Commands, RPC requests or responses, service errors, and arbitrary traffic SHALL NOT traverse a cross-VC replication edge unless the service separately defines explicit proxy semantics, authority, correlation, failure behavior, and bounds. For control-relevant state, original provenance SHALL be representable and retained. Configuration and runtime validation SHALL reject competing producers for a destination `ProducerKey` unless the service defines explicit authority and arbitration. A cross-VC replicator SHALL NOT silently masquerade as the original producer.

Device-wide status published over several healthy VCs is one example of intentional cross-VC publication. It is not evidence that those VCs form one route.

8.3 Multicast and groups

Deferred: the initial prototype defines no generic multicast wire request, no generic group-Index range, and no dynamic group membership. The multicast control bit remains zero/unassigned.

Initial one-to-many behavior is limited to:

- physical link visibility plus configured passive listeners;
- separately configured transmit/publication instances; or
- explicit bounded cross-VC replicators.

Multicast/group encoding, fan-out ownership, producer attribution, error behavior, and reliable interaction require a later specification.

9. Static route and configuration validation

Each VC SHALL have a deterministic, statically generated forwarding topology. Physical loops MAY exist, but the active forwarding graph for every configured VC and target SHALL be loop-free. The initial protocol does not add TTL, spanning-tree negotiation, route discovery, or duplicate-path suppression to repair invalid configuration at runtime.

Build-time tooling SHALL reject, at minimum:

- duplicate individual Index assignments within a VC;
- undefined or illegal VCN, Index, namespace, EndpointId, or symbolic reservation use;

- missing next hops and unreachable configured targets;
- forwarding cycles and unintended ambiguous replication;
- `kLocalHost` paths that touch an external LLL;
- cross-VC entries represented as transparent routes;
- ingress paths not authorized for the VC;
- use of `UpstreamHost::kLocalHost` without `VirtualCircuit::kLocalHost`, or either local marker on external ingress or egress;
- inconsistent endpoint-alias mappings, unknown alias digests, and mismatched alias digests for alias-bearing profiles;
- learned-from-ingress enablement on unauthenticated multi-access links or outside its configured peer/origin allowlist;
- competing producers for one `ProducerKey` without explicit authority and arbitration;
- incomplete or overbroad cross-VC replication edges;
- LLL/profile, MTU, descriptor-field, and transport capability mismatches.

Every deployed node SHALL expose a version or fingerprint covering the network-relevant static configuration, plus the capabilities needed to validate its configured paths. Fingerprints need not appear in every application message. Management/diagnostic services SHOULD compare expected and observed fingerprints and capabilities at integration time and runtime.

A mismatch SHALL NOT cause dynamic route creation or silent reinterpretation. The affected operation SHALL fail closed or enter an explicitly configured degraded mode, with a local diagnostic record.

10. Error handling, congestion, and recovery

Receivers and routers SHALL validate descriptors, endpoint configuration, route authority, reserved values, profile capabilities, and message bounds before dispatch or forwarding. Invalid traffic SHALL be dropped or rejected at the detecting boundary and counted by a stable local reason.

Core reason categories SHOULD include malformed descriptor, unknown namespace or endpoint, unknown VC or target, no route, unexpected ingress, unsupported transport/profile, capability or fingerprint mismatch, prohibited local-marker ingress or egress, producer conflict, congestion, and reassembly/link failure. Exact enums and wire reporting are defined elsewhere.

Remote infrastructure-error reports are optional. When enabled, they SHALL be bounded, rate-limited, aggregatable, and lower in scheduling importance than critical service traffic. An error report SHALL NOT trigger another infrastructure-error report. Local counters remain authoritative even when a remote report is suppressed or cannot be sent.

Transmit congestion and a temporarily unavailable route/link are normal runtime outcomes. A transmit API SHALL report non-acceptance explicitly. Services SHALL define bounded retry, replacement, queueing, or drop behavior and SHALL NOT advance protocol state as though an unaccepted message had entered the transport.

A link or Link Entity MAY restart without resetting the node. Restart SHOULD discard uncertain transient framing, reassembly, queue, and hop-local state while retaining static configuration, persistent counters, latched fault information, and defined higher-layer ownership. Services and transports own any required

conversation recovery. Health/status publication over an unaffected VC is an explicit publication operation, not automatic rerouting.

11. Core invariants

1. The canonical working descriptor is 8-bit Control, 16-bit Routing, and 16-bit `EndpointId`; profile encodings must reconstruct it.
2. Endpoint identity is `(Namespace, EndpointId)`, with services using literal IDs `0x0100..0xFFFF`.
3. Low `EndpointId` aliases are private to one LLL and never cross its upper boundary.
4. Direction selects the destination service role, not physical movement.
5. A downstream endpoint may transmit asynchronously without reversing roles.
6. `UpstreamHost` normally identifies a configured Main and is not a general node address; its reserved local marker is valid only with the reserved local VC marker and never crosses an external LLL.
7. The complete downstream node target is `(VCN, Index)`.
8. Index is VCN-scoped; the packet identifies a VC and member while static configuration contains the physical route.
9. Paired `UpstreamHost::kLocalHost` and `VirtualCircuit::kLocalHost` traffic never reaches an external LLL; the symbols have distinct field types and provisional numeric values.
10. Main-to-Main traffic uses `kMainCompute`; its Index relationship to `UpstreamHost` remains unresolved.
11. Arbitrary ECU-to-ECU unicast is not a core routing capability.
12. Passive observation does not create a subscription, peer, producer, reply or forwarding authority, acknowledgment obligation, or delivery guarantee.
13. Same-VC transparent forwarding preserves canonical identity, routing, transport state, and payload and never changes producer or peer.
14. Cross-VC replication is explicit publication origination, not routing, and each edge is constrained by message type, producer authority, provenance, and conflict policy.
15. Each VC's configured forwarding graph is loop-free and validated before deployment.
16. The initial prototype has no generic multicast request and uses symbolic QoS classes without numeric ordering assumptions.
17. Configuration disagreement is diagnosed; it never authorizes dynamic reconfiguration or guessed semantics.

12. Open issues

The following are explicit specification gaps, not implementation latitude:

1. Freeze packed bit positions, byte order, reserved-bit policy, and named canonical/wire profile versions.
2. Assign numeric values separately for `UpstreamHost::kLocalHost`, `VirtualCircuit::kLocalHost`, `kMainCompute`, transport selectors, Direction, and symbolic QoS classes.
3. Define the exact relationship, if any, between `UpstreamHost` and `kMainCompute` Index.
4. Complete source/origin and return-path semantics without weakening the accepted downstream target `(VCN, Index)` or turning `UpstreamHost` into a general address.

5. Specify the `ReplyContext` API, lifetime representation, authorization, and behavior across endpoint, router, and Link Entity restart.
6. Freeze neither `ProducerKey` nor source/origin metadata as a wire field; define any later interoperable provenance representation for duplicate EndpointIds, capture, passive listeners, and cross-VC attribution.
7. Standardize QoS symbols, numeric mapping, scheduling contracts, and link-specific arbitration behavior.
8. Decide whether and how multicast/group addressing is added after the unicast/publication prototype, including interaction with producer identity and transport semantics.
9. Define configuration-fingerprint coverage, capability schema, comparison protocol, and degraded-mode policy.
10. Define immutable conformance profiles and vectors in the link/transport and CAN specifications.
11. Specify Reliable Transport separately when validated by a concrete service.

Appendix A. Non-normative routing examples

A.1 Main command and asynchronous ECU status

Main A's upstream endpoint sends a command to the downstream endpoint at `(MotionVC, MotorEcuIndex)`. The ECU later sends status to the upstream role without changing either endpoint's role. `UpstreamHost` identifies Main A; it does not address the motor ECU.

A.2 Passive same-VC observation

Steering ECU publishes state toward its configured Main peer on `ChassisVC`. Brake ECU can see the frame on the shared link and has a static passive-listener subscription. It consumes the value without acknowledging it or becoming Steering ECU's peer. Its listener retains Steering ECU's validated `ProducerKey` and gains no reply or forwarding authority.

A.3 Transparent heterogeneous continuation

A message on `ControlVC` crosses CAN, a gateway, and UART. The gateway changes LLL framing only. VCN, Index, endpoint identity, roles, transport state, and payload remain unchanged, so the gateway is neither producer nor peer.

A.4 Explicit cross-VC status replication

An ECU status publication is observed on `PowertrainVC`. A configured cross-VC replicator emits a separate summary on `TelemetryVC`. Its edge names the source and destination `ProducerKey` values, eligible status message type, provenance, authority owner, and conflict policy. The second message is an explicit publication under `TelemetryVC` policy, not a route continuation.

A.5 Main-to-Main service

Main A's upstream service instance addresses Main B's downstream instance at `(kMainCompute, MainBIndex)`. Main B replies using request-scoped context. Configuration treats `MainBIndex` and Main B's `UpstreamHost` identity as distinct until their relationship is standardized.

Application Protocols and Services

Status: Consolidated design contract

Maturity: Provisional; suitable for prototype alignment, not a frozen API or wire specification

Scope: Bounded application protocols, service instances, local endpoint behavior, and brokerless pub/sub

This document owns application/service semantics for the embedded message fabric. [Core protocol and routing](#) owns canonical addressing and Virtual Circuits (VCs); [logical links and transports](#) owns delivery and link contracts; [prototype and validation](#) defines how the requirements below are proved. The [architecture overview](#) and [README](#) define the wider scope and authority.

The shapes shown here are semantic examples. They are not final APIs, field layouts, or wire encodings.

1. Role-directed protocols

Every service protocol defines two stable roles:

- **Upstream** originates stimulus, requests, commands, or orchestration.
- **Downstream** receives that stimulus and performs the responder or controlled role.

The roles belong to the service instance, not to physical topology or device class. An ECU may be upstream for fault reporting while a Main logger is downstream. A downstream endpoint may reply immediately, publish feedback later, or originate substantial asynchronous traffic without reversing the roles.

A genuinely symmetric exchange is composed from two directed service instances, one in each direction. A convenience wrapper may present them as one duplex facility, but the underlying authority, routing, and state remain explicit.

2. Endpoint identity and service instances

The canonical endpoint identity is:

```
(namespace: 2 bits, endpoint_id: 16-bit literal)
```

Namespaces `0..2` are deployment-defined. Namespace `3`: **open ecosystem namespace; allocation governance remains TBD**. Canonical literal endpoint IDs are `0x0100..0xFFFF`.

Values `0x0000..0x00FF` are reserved for optional link-local aliases. An LLL may map a configured alias to a literal ID on receive and reverse the mapping on transmit, but aliases never reach the Service Router, a transport, a service, capture in canonical form, or application code. Alias configuration does not change endpoint identity.

An alias-bearing profile fails closed unless both peers derive from the same generated topology artifact or use compatible alias digests provisioned by an out-of-band mechanism. An unknown or mismatched digest disables the alias profile for the affected peer or link, and traffic under that profile remains disabled. A separately configured literal-ID profile may be used instead, but there is no automatic fallback or profile negotiation.

`UpstreamHost` normally identifies a configured Main identity. Local IPC/ICC instead uses the reserved symbolic `UpstreamHost::kLocalHost`, which is not a node address and is valid only with `VirtualCircuit::kLocalHost`. These symbols belong to different field types and their numeric values remain provisional. A local marker is rejected on external ingress or egress, and either local symbol without the other is invalid. Source/origin and return-path semantics remain explicitly open; this document does not infer them from `UpstreamHost`.

An endpoint ID identifies a receiving application-protocol endpoint, not a device, route, service name, or software build. A protocol may use several endpoints, and a deployment may instantiate the same protocol several times under different literal IDs. A service instance therefore binds, at minimum:

- one directed protocol role;
- one or more literal endpoint identities;
- a declared transport and size profile;
- configured peer/routing policy;
- bounded RX, TX, and local-delivery behavior;
- service-specific authority, freshness, and error rules.

Default IDs supplied by a reusable library are conveniences only. The deployment owns final allocation and collision checks.

3. Five peer-binding modes

Each endpoint registration declares exactly one of five bounded binding modes. These modes describe where transmission context comes from; they do not create general routing.

1. **Static binding** — uses one configured peer, or a fixed bounded set selected by application policy, for replies and autonomous transmission.
2. **Learned-from-ingress binding** — retains a finite configured number of ingress-derived peers for later transmission, but only under an enabled authorization mode below.
3. **Request-scoped binding** — may reply only through opaque context supplied with a valid incoming request. The context is bounded, cannot be synthesized by service code, and expires at a defined scope boundary.
4. **Receive-only** — accepts configured traffic and never transmits protocol messages. A passive listener is the common case.
5. **Transmit-only** — publishes through a statically configured destination or VC and does not acquire a peer from ingress.

Learned-from-ingress binding is disabled on unauthenticated multi-access LLLs. It may be enabled only on a statically single-peer ingress or under a named authenticated-origin profile, and only within a preconfigured allowlist. The allowlist constrains permitted origin, service, VC, Index, and interface values. Route, VC, Index, interface, and descriptor validation are not authentication. Every enabled learned mode defines finite capacity, expiry, replacement/eviction, restart invalidation or restoration, and auditable learn, replace, expire, reject, and clear events. Prototype conformance tests all enabled binding modes; learned-from-ingress mode is not assumed to be unconditionally enabled.

Protocols needing richer multi-peer sessions maintain explicit bounded session state above these primitives. They must not treat arbitrary recent traffic as permission to transmit.

Replies and autonomous downstream transmission

The receive path supplies parsed service data separately from an opaque reply context. A request-scoped responder can return a success or service error without interpreting raw `VCN`, `Index`, direction, or `UpstreamHost` fields.

Delayed work must make the lifetime rule explicit. If a response can outlive the callback or dispatch operation, the infrastructure must provide a bounded retained reply handle or require another binding mode; retaining references to transient ingress state is invalid.

Request-scoped context authorizes only the corresponding response flow. Periodic feedback, completion events, unsolicited alarms, and other autonomous downstream messages require a static or learned-from-ingress binding configured for that purpose. A successful request must not silently install a durable peer unless the endpoint registration is explicitly configured for learned-from-ingress binding.

A protocol requiring both request-scoped replies and autonomous transmission uses separate endpoint registrations or explicit bounded named sub-bindings that each have one declared binding mode, authority, and lifetime. Separate registrations are recommended for the initial prototype. This is a composition rule, not frozen API spelling.

4. Service message contracts

The default service-message prefix is semantically:

```
u8 protocol_version
u8 message_type
```

`protocol_version` identifies an incompatible interpretation of the service protocol. It does not change for an implementation-only hotfix. `message_type` distinguishes requests, successful responses, service errors, snapshots, feedback, and events and also helps detect endpoint/profile misconfiguration. A separate magic field is optional, not standard.

Each serious service is defined by a canonical schema over bytes, not by a C/C++ ABI. Its specification must state:

- exact serialization, byte order, bit numbering, signed representation, and length interpretation;
- every field width, range, enum value, and invalid value;
- engineering units, scale, offset, clock domain, and validity metadata where applicable;
- transmitter values and receiver behavior for padding and reserved fields;
- required, optional, and safely ignorable message types;
- malformed-input, unknown-version, unknown-type, and unsupported-feature behavior;
- freshness, sequence, correlation, duplicate, and restart semantics where applicable.

Generated classes and structs are views or codecs derived from the schema; native padding, enum layout, alignment, and endianness never define the wire contract. Decoders must safely handle unaligned data and reject lengths before field access. Bounded arrays, strings, and opaque byte sequences are allowed; unbounded MCU allocation is not.

Sub-byte integers, explicitly sized enums, fixed-point values, and reserved fields are first-class schema features. Compact encoding is encouraged when it preserves clarity and is justified by a constrained profile, but this document does not select a final schema language or serialization format.

Version, compatibility, and build identity

Three identities remain separate:

- **Protocol version** — the major wire-semantic interpretation carried in each service message.
- **Compatibility fingerprint** — a stable, preferably generated fingerprint over the canonical schema and all communication-relevant profile/configuration choices. DEETS or equivalent management infrastructure may expose it.
- **Build identity** — the exact software build, reported by BIO or equivalent.

An internal hotfix may change build identity without changing compatibility. A wire-visible build option changes compatibility even if the protocol version remains valid. An incompatible redesign changes protocol version and compatibility. A build hash must not be used as a substitute for schema compatibility.

5. Size and profile rules

Services use one of three sizing patterns:

- **Fixed-small** — every message has a modest fixed maximum, preferably efficient on constrained links.
- **Bounded-profile** — a finite named profile changes capacities such as maximum entries, samples, or string bytes while preserving message meaning and state-machine behavior.
- **Bulk-transfer** — small control/status messages coordinate a separately bounded segmented or object-transfer facility.

A profile may alter capacity, batching, or storage bounds. It must not silently remove semantic fields, redefine units, or change protocol behavior merely to fit a link. Published profiles are named, immutable, and independently testable; incompatible changes require a new profile identity.

Every service instance declares maximum encoded message size, supported transport profiles, expected and maximum rates, burst depth, freshness/deadline requirements, and compatible LLL capacities. Broad portability does not require every service to fit every link.

Service and deployment configuration refer to QoS only by symbolic class name. Numeric ordering, numeric priority comparisons, and final symbolic-to-numeric mapping remain open.

Classical CAN remains useful design pressure, especially at small aggregation depths, but its consolidated PDU format is unfinished. Capacity estimates from draft CAN layouts are not stable application contracts; see the [experimental CAN PDU adapter specification](#).

6. Bounded local endpoint behavior

Network delivery and local concurrency are separate policies selected for each service instance.

Latest-value snapshots

A latest-value endpoint retains one coherent decoded snapshot. One receive/publish writer may update it while multiple local readers observe either the previous or next complete value, never a torn mixture. Suitable implementations include a seqlock or double buffer, subject to the platform memory and cache-coherency contract.

Snapshots carry enough metadata for consumers to assess validity: receive or source sequence, time/age basis, producer restart epoch where required, and decode/E2E status where available. A snapshot is not an event history. Replacing an unread value is normal and is counted when diagnostically useful.

Bounded event queues

An event endpoint uses a fixed-capacity FIFO or ring when every retained event matters. It defines:

- producer and consumer concurrency;
- ordering and sequence/gap behavior;
- overflow policy: reject newest, drop oldest, coalesce defined events, or enter a fault state;
- observable overflow and high-water counters;
- restart and retained-state behavior.

Queue capacity is a deployment/profile parameter with a static resource cost. "Queue until memory is available" is not a valid MCU policy.

Single-writer and multi-caller TX

A **single-writer TX endpoint** is owned by one task, thread, core, or serialized execution context. A **multi-caller TX endpoint** adds a bounded thread-safe funnel in front of the same protocol endpoint.

Multi-caller support must define arbitration, ordering, per-call result, ISR allowance, wakeup behavior, and storage ownership. It must not convert an application endpoint into an unbounded multi-producer queue. When ordering or caller authority affects service meaning, those semantics belong to the service protocol rather than being hidden in the funnel.

7. TX rejection and congestion

Transmit submission can fail during normal operation. Every transmitting service defines bounded behavior for local rejection and does not busy-spin.

Candidate result categories include accepted, replaced/coalesced, congested, invalid route, invalid message/profile, and link unavailable. Names and API representation are provisional. Local rejection means the message never entered the transport; a later reliable timeout would mean that transport had accepted ownership. These states must not be conflated.

Required service policies:

- **Latest value:** retain at most one pending value; a newer value may replace it; retry on notification, tick, or timer.
- **Command or high-integrity operation:** expose rejection to the caller or retain the exact bounded operation; do not advance protocol state before the defined commitment point.

- **Event stream:** enqueue in a bounded FIFO and apply the declared overflow policy.
- **Best-effort diagnostic:** drop, aggregate, or suppress under congestion and increment counters.
- **DRIP detail response:** retain a bounded cursor, pause generation, and resume when capacity returns.

Successful local enqueue is the earliest normal TX-state advancement point. A protocol may deliberately require a later point, such as service acceptance, but Reliable transport and its commitment rules remain TBD.

The LLL/router path should expose bounded capacity notification and metrics such as occupancy, high-water mark, rejection, drop, replacement, and completion timestamp where supported. Services must remain correct if notifications are coalesced or delayed.

8. Pub/sub and listeners

Pub/sub is brokerless and statically configured:

- each publication is associated after ingress/routing validation with an opaque deployment-local `ProducerKey` assigned by static configuration for prototype, configuration, and capture use;
- one authoritative producer originates for each `ProducerKey` unless an explicit authority/arbitration protocol says otherwise;
- any number of configured local consumers may read a shared snapshot or event queue;
- any number of statically configured passive listeners may consume an explicitly authorized publication visible on an attached ingress link without becoming addressed peers;
- a configured transparent forwarder may preserve and carry an authorized publication on the same VC to another attached link;
- a configured cross-VC replicator may consume an eligible publication on one VC and originate a distinct publication on another VC.

`ProducerKey` is runtime metadata, not a frozen wire field. For received traffic it is derived only after ingress and routing validation succeed; a locally originating endpoint receives only the key assigned to its static registration. Endpoint identity and physical visibility alone do not establish producer identity. A listener retains the validated `ProducerKey` with consumed data.

A listener does not acknowledge, control flow, acquire reply or forwarding authority, or gain delivery guarantees. Physical visibility and endpoint ID do not create a subscription. Same-VC forwarding can carry a publication to another attached link; listening alone cannot. Physical bus broadcast does not imply network-wide broadcast.

Every cross-VC replication edge is explicit, bounded, observable, and loop-free, and declares eligible publication message types, source `ProducerKey`, destination `ProducerKey` and authority owner, provenance representation, and conflict policy. Commands, RPC requests or responses, service errors, and arbitrary traffic are forbidden on such an edge unless the service defines explicit proxy semantics, authority, correlation, failure behavior, and bounds. Control-relevant state requires representable original provenance. Configuration and runtime validation reject competing producers for a destination `ProducerKey` unless an explicit authority/arbitration protocol resolves them. Cross-VC replication does not create general ECU-to-ECU routing.

Dynamic subscriber discovery, runtime route creation, a mandatory broker, and implicit fan-out inferred from endpoint identity are outside the base model. See [core protocol and routing](#) for listener, forwarding, and

replication distinctions.

9. Service-level errors and diagnostics

Service-level failure belongs to the service stream. A protocol may define an error-response message alongside request and success types, optionally using a reusable compact representation such as BRO. Useful categories include malformed request, unsupported version/type/operation, invalid argument or state, permission denied, busy, and rejected. Correlation data is included only where the protocol semantics require it.

Infrastructure faults such as no route, unknown endpoint, transport mismatch, malformed envelope, reassembly failure, and unexpected ingress are recorded by generic metrics/diagnostics even when the application endpoint is receive-only or does not implement error messages. They are not automatically converted into service responses.

Diagnostic containment is mandatory:

- an error report never recursively generates another remote error report;
- local counters and latched evidence remain the source of truth;
- remote reporting is optional, bounded, rate-limited, and normally lower priority than control traffic;
- repeated errors are aggregated and suppressed counts remain observable;
- malformed or babbling peers cannot force unbounded diagnostic work;
- ordinary endpoints do not receive every other node's errors.

DRIP is the preferred pattern: publish a small bounded summary periodically, then return larger bounded detail only on explicit request. Loss of the remote diagnostic path must not erase local evidence.

10. Safe use of the service catalog

The catalog in the source drafts is a set of candidates, not an automatic standard library or allocation authority. A service should be promoted only when it has a clear semantic boundary, bounded schema, explicit privilege model, compatibility metadata, resource profile, test vectors, and at least one real deployment.

Useful catalog guidance:

- MENU describes configured endpoint instances; it does not enable them or create routes.
- DEETS reports communication compatibility; BIO reports build identity; WHO reports hardware identity.
- CAPS, if retained, selects among finite precompiled profiles; it does not negotiate arbitrary encodings.
- TABS is suitable for disposable scalar prototypes. Features graduate to dedicated services when they need transactions, timing, richer state, or interaction.
- BRO is an optional service-error shape, not a substitute for local infrastructure diagnostics.
- TELL/DRIP-style logs and diagnostics are bounded and best effort; they never block critical work indefinitely.
- SUS, AMA, ROSE, fault injection, firmware update, raw capture/injection, and similar dangerous or privileged services are outside the initial prototype. They require a separate development build, build-time enablement, and local-only, non-forwardable, allowlisted operations. Runtime configuration alone cannot enable them.

Catalog names remain provisional until their contracts and endpoint allocations are published. The local/private security posture and gateway risks are discussed in [rationale](#), [use cases](#), and [risks](#).

11. Required declaration for each service

Before a service is considered implementable, its specification records:

- roles, authority, binding mode, and permitted autonomous traffic;
- endpoint instances and literal-ID allocation policy;
- message types, canonical schema, units, reserved fields, and serialization;
- protocol version, compatibility inputs, and profile identity;
- state, event, command, RPC, or object semantics;
- size/rate/burst bounds, freshness, ordering, loss, duplicate, and restart behavior;
- local snapshot/queue/concurrency policy;
- TX rejection, congestion, timeout, and overflow behavior;
- service-error and nonrecursive diagnostic behavior;
- transport/LLL compatibility and resource budget;
- passive listener, transparent forwarder, cross-VC replicator, `ProducerKey`, provenance, capture, and security policy.

Reliable transport is wholly TBD. CAN and HDLC are experimental until their owning documents define immutable profiles and golden vectors. No service may claim reliable or cross-link wire interoperability merely by referring to an exploratory draft.

Logical Links and Transports

Status and authority

This document defines the **stable generic software contracts** for Logical Link Layers (LLs), Hardware Links, and the currently supported transport vocabulary. It does not stabilize any particular wire encoding. The overall layering and scope are summarized in [Architecture Overview](#).

Wire interoperability exists only under a separately named, immutable profile. The serial candidate is documented in [HDLC Logical Link Profile](#) and is explicitly experimental. Link ownership, restart, and device-wide status are defined in [Link Entity Runtime and Status](#).

The canonical descriptor, endpoint identity, virtual-circuit semantics, routing, and forwarding rules belong to [Core Protocol and Routing](#). Application delivery policies belong to [Application Protocols and Services](#). This document uses those contracts rather than redefining them.

Normative terms such as **shall** and **must** apply to the stable generic contracts unless a section is marked Provisional, Experimental, Non-normative, or Open.

1. Layer boundaries

```
graph TD; A[Application / Service] --- B[Endpoint / Service Router]; B --- C[Transport]; C --- D[Logical Link Layer]; D --- E[Hardware Link];
```

The boundaries are:

- A **transport** adds delivery semantics to one bounded application message.
- An **LL** maps one canonical PDU value to and from one configured link profile.
- A **Hardware Link** moves profile-specific bytes, frames, datagrams, or equivalent transfer units. It has no endpoint or service semantics.
- A **Link Entity (LE)** owns and serializes the mutable runtime for one or more LLL and Hardware Link instances. An LE does not own an entire virtual circuit; see [Link Entity Runtime and Status](#).

Physical signaling, transceivers, modulation, and electrical characteristics are outside the generic contracts.

2. Canonical bounded-PDU service

2.1 Canonical PDU value

At the transport/LLL boundary, one PDU is a discrete semantic value containing:

- the canonical descriptor and routing metadata defined by the core protocol;
- any transport-specific header or status;
- one bounded application payload.

The canonical PDU value is not necessarily a contiguous byte array, and it is not necessarily identical to a wire header. A link profile may encode canonical fields into link-native metadata, omit fields that static configuration reconstructs unambiguously, or serialize the complete value. Receive processing shall reconstruct the same canonical value before delivery upward.

An LLL shall not reinterpret application payload bytes. It shall not terminate end-to-end transport state.

2.2 Message boundaries and bounds

The LLL service is message-oriented:

- One accepted TX request represents exactly one complete PDU.
- One successful RX delivery represents exactly one complete, link-validated PDU.
- Partial PDUs shall not be delivered upward.
- The generic LLL does not segment an arbitrarily large PDU. A profile-specific adapter may fragment and reassemble internally only when that behavior is part of the named profile.
- Every configured instance shall expose finite TX and RX PDU bounds.
- Length and profile compatibility shall be checked before an implementation reads or writes outside supplied storage.

The effective application-payload limit depends on the service, transport overhead, canonical metadata encoding, and selected LLL profile. Static configuration tooling should reject combinations that cannot fit.

2.3 Receive delivery is not callback-dependent

The stable contract requires a bounded receive-delivery port, not a mandatory callback ABI. Valid implementations include:

- a consumer-owned `try_receive`/drain interface;
- a bounded ownership-transfer queue;
- direct publication into an explicitly configured endpoint storage object;
- an optional short callback adapter layered over one of those mechanisms.

No service or LLL profile may require arbitrary application code to run in the LLL execution context. If callbacks are offered, their execution context, work bound, reentrancy rule, and ownership result shall be explicit.

2.4 TX acceptance and completion

A TX submission shall return promptly with an explicit result. At minimum, results shall distinguish:

- accepted into bounded LLL-owned state;

- rejected because capacity is temporarily unavailable;
- rejected because the route/profile is invalid;
- rejected because the PDU or length is invalid;
- rejected because the link is disabled or faulted.

Acceptance means that ownership has transferred as documented and the request entered the local transport/link path. It does **not** mean peer delivery, successful physical transmission, or acknowledgment.

After acceptance, the implementation shall eventually do one of the following:

- complete transmission and release/report the accepted object;
- report cancellation caused by an explicit stop or restart;
- report a terminal local fault.

Completion notification may be polled, queued, or delivered through an optional bounded callback. The implementation shall define whether completion means queued to hardware, consumed by hardware, or physically completed. The generic contract does not equate any of those events with end-to-end delivery.

3. Ownership and borrowing

Every PDU buffer has one logical owner at a time.

Two API forms are permitted:

- **Borrowed view:** immutable for a documented call or lease scope. The borrower shall not retain it after that scope or mutate the backing storage.
- **Owned handle:** exclusive ownership is transferred explicitly. The receiver may retain or enqueue the handle and shall eventually release or transfer it.

Requirements:

- Ownership transfer shall be mechanically visible in the API.
- A rejected TX submission leaves ownership with the caller.
- An accepted owning TX submission transfers ownership to the receiving layer.
- A borrowed TX submission may be accepted only if the implementation copies the required bytes before return or documents a bounded lease extending through a later completion event.
- RX storage retained by application code shall remain valid until released, including across task/core transfer and LE restart. The storage allocator therefore needs a lifetime broader than restartable LE state.
- Buffer reclamation shall occur through the allocator's defined owner or synchronization mechanism; arbitrary contexts shall not mutate a shared free list.
- Mutable access after publication is prohibited unless a separate, explicit protocol transfers ownership back.

Zero-copy operation is desirable but not required. Correct ownership and bounded behavior take precedence.

4. Stable Logical Link contract

For each configured instance, an LLL shall:

1. accept or reject complete canonical TX PDU values without unbounded waiting;
2. encode accepted values according to one configured named profile;
3. consume Hardware Link RX units incrementally;
4. validate profile framing, lengths, and link-level integrity before parsing untrusted PDU fields;
5. reconstruct canonical PDU values and deliver only complete valid values upward;
6. expose bounded progress, drop, parse, integrity, oversize, congestion, and lifecycle diagnostics;
7. support explicit start, stop, reset, and fault handling through its owning LE;
8. preserve message ordering only to the extent promised by the selected profile and Hardware Link;
9. keep application/service dispatch outside Hardware Link drivers;
10. avoid hidden dynamic allocation requirements.

An LLL may exploit link-native addressing or priority metadata, but the resulting canonical value shall obey the core protocol. A transparent forwarding path is compatible only when every hop can preserve or unambiguously reconstruct that value.

5. Stable Hardware Link contract

A Hardware Link supplies the smallest useful transfer service below an LLL. Its profile-specific RX/TX unit may be a byte span, frame, datagram, FIFO entry, DMA descriptor, or shared-memory record.

Each instance shall provide:

- bounded RX acquisition or draining;
- bounded TX acceptance with explicit temporary rejection;
- explicit transfer-unit lengths and capacity;
- lifecycle control or integration with equivalent driver lifecycle control;
- fault and progress observations;
- a way to wake or notify the owning execution context when useful;
- ownership and completion semantics for every submitted storage object.

A Hardware Link shall not:

- parse endpoint IDs, transport headers, or application payloads;
- dispatch services;
- claim end-to-end delivery;
- silently retain caller-owned storage beyond its documented lease;
- invoke unbounded application work from an ISR.

5.1 Capability description

Each configured Hardware Link and LLL shall expose an immutable capability description sufficient for static composition. Relevant capabilities include:

- RX and TX transfer-unit kind;
- maximum RX and TX unit size;
- maximum decoded/reassembled PDU size;

- required alignment and contiguous-storage constraints;
- scatter/gather and zero-copy support;
- full-duplex, half-duplex, or simplex behavior;
- ordering and possible duplication behavior;
- link-provided error detection, if any;
- whether TX completion and timestamps are available;
- whether DMA, ISR production, cross-core access, or cache maintenance is involved;
- bounded queue depth or other acceptance limits.

Capabilities describe the local implementation. They are not a substitute for a wire profile and do not imply runtime negotiation.

6. Named immutable profiles and compatibility

Every interoperable LLL configuration shall select a named, versioned profile. A stabilized profile is immutable. Any incompatible change requires a new name/version.

A profile shall specify all interoperability-relevant behavior, including:

- exact field and transfer-unit layout;
- byte and bit order;
- framing, escaping, padding, and reserved-value rules;
- PDU length interpretation and all minimum/maximum bounds;
- integrity algorithm, field order, coverage, and validation order;
- fragmentation/reassembly and timeout behavior, if present;
- mapping between canonical fields, static configuration, and link-native metadata;
- malformed-input and resynchronization behavior;
- required diagnostics;
- representative valid, boundary, and invalid conformance vectors.

Peer compatibility is a static deployment property. Peers shall use the same named profile and compatible bounds/configuration. Profiles are not auto-detected from traffic, and the base architecture does not require profile negotiation.

Static validation should reject:

- profile-name/version mismatch;
- incompatible PDU bounds;
- canonical fields that cannot be preserved or reconstructed;
- incompatible aliases or static mappings;
- service/transport messages that exceed the effective capacity;
- unsupported Hardware Link capability requirements.

7. Execution and concurrency

7.1 One serialized mutable LLL context

Each LLL instance has exactly one logical execution context that mutates parser state, reassembly state, TX scheduling state, timers, and its owned queues/pools. It may run in a bare-metal loop, a dedicated task, or another explicitly serialized executor.

Other tasks or cores shall communicate through bounded submission ports, ownership-transfer queues, immutable snapshots, or explicitly synchronized control requests. They shall not concurrently call arbitrary state-mutating LLL operations.

Read-only status may be exposed concurrently only through a coherent snapshot or other documented synchronization mechanism.

7.2 Interrupt context

ISR work should be limited to:

- acknowledging hardware;
- capturing a bounded RX unit or completion record;
- publishing it to the owning context;
- waking that context.

Parsing complete PDUs, endpoint dispatch, arbitrary callbacks, blocking, and general allocator/free-list manipulation shall not occur in an ISR unless an implementation defines and verifies a stricter ISR-safe profile. Such an exception does not weaken the generic contract for other implementations.

ISR-to-task publication requires the platform's documented atomic and memory-ordering operations.

`volatile` alone is not a synchronization primitive.

7.3 Cross-core execution

Cross-core transfer shall use an explicit publication boundary. Before publishing an owned handle, the producer completes all payload writes and performs any required cache clean/release operation. The consumer performs the matching acquire/invalidate operation before reading.

The queue or mailbox implementation shall define:

- producer and consumer identities;
- ordering and atomic-width assumptions;
- cache/coherency requirements;
- full/empty behavior;
- reset behavior while records are in flight.

An implementation may copy when zero-copy ownership cannot be made safe.

8. `kLocalHost` semantic equivalence

`VirtualCircuit::kLocalHost` selects the reserved local-only communication domain.

`UpstreamHost::kLocalHost` is the required companion marker for that domain; it does not select the domain by itself. Implementations shall reject either local symbol without the other, and shall never emit either on an external link or accept either from external ingress.

Same-core and cross-core `kLocalHost` paths shall provide the same externally visible protocol semantics:

- the same canonical PDU value and endpoint identity;
- the same service directionality and configured peer binding;
- the same transport interpretation;
- the same semantic validation and configured local routing policy;
- explicit bounded acceptance and backpressure;
- the same distinction between accepted, rejected, delivered, and consumed.

The implementation mechanism may differ. A same-core path may use direct serialized dispatch; a cross-core path may use shared memory, a FIFO, a queue, or a copy. Optimization shall not make same-core delivery reentrant when the equivalent cross-core path is deferred, unless that execution difference is an explicit endpoint policy.

`kLocalHost` guarantees off-host containment only. The marker does not authenticate callers or isolate tasks, processes, or cores. Authorization requires a platform principal and enforcement boundary, such as MPU/MMU permissions or authenticated IPC. Without such a boundary, all code able to submit to the local path is trusted. Bounds, descriptor, endpoint/service-direction, peer-binding, payload/schema, and routing validation still apply, but those checks shall not be represented as caller authorization. Local corruption protection may be omitted only as a named local profile decision.

9. Snapshot and queue semantics are separate

Storage policy is above the generic LLL contract:

- A **snapshot** retains the latest complete value. A newer publication may replace an older unread value. Readers observe a coherent value but are not promised every update.
- A **queue/event stream** retains discrete accepted events in order up to a bounded capacity. Overflow has an explicit reject, drop, or replacement policy.

Neither may masquerade as the other. In particular:

- a snapshot replacement is not a queue delivery;
- queue exhaustion shall not silently become latest-value replacement;
- snapshots and live status require coherent publication;
- event, command, and reliable-protocol state normally require non-replacing storage.

Callbacks are only notification mechanisms and do not define storage or ownership semantics.

10. Congestion and bounded backpressure

Temporary TX rejection is a normal runtime condition. No caller may assume submission succeeds, busy-spin indefinitely, or advance protocol state as though a rejected PDU entered the link.

Every transmitting service/transport shall define bounded behavior, such as:

- retain and replace one pending latest value;
- retain exact protocol state and retry later;
- use a bounded FIFO with a defined overflow result;
- drop or aggregate best-effort diagnostics while incrementing a counter.

An LLL should expose bounded capacity/congestion observations such as available slots, occupancy, high-water mark, rejected count, dropped count, and replaced count. Notification of newly available capacity may be polled, scheduled, queued, or callback-based; it is not required to be a callback.

Local TX rejection is distinct from a transport timeout: rejection means the PDU never entered the transport/link path, while a timeout applies after transport acceptance.

11. Transport profiles

11.1 Unreliable Datagram — stable and completed

Unreliable Datagram is the only completed transport contract in the initial architecture.

It provides:

- one bounded discrete message per PDU;
- no transport header;
- no retransmission or acknowledgment;
- no connection state;
- no delivery, uniqueness, or ordering guarantee beyond what lower layers happen to provide.

Successful local acceptance does not imply remote receipt. Applications that need freshness, duplicate handling, transactions, or retry shall define those semantics themselves or select a future standardized transport.

11.2 Sequenced/E2E-Protected Datagram — Provisional

Sequenced/E2E-Protected Datagram is a candidate future transport. Candidate metadata includes an alive/sequence counter and an end-to-end check value. Exact fields, coverage, state, reset behavior, acceptance window, diagnostics, and profile ID are unresolved.

No implementation shall claim interoperability from this document. A future profile must define all serialization and conformance vectors before stabilization.

E2E integrity is distinct from LLL integrity:

- LLL integrity detects corruption in one link/profile transfer or reassembly.

- E2E integrity protects selected producer-to-consumer semantics across transparent forwarding and re-encapsulation.

Neither mechanism provides cryptographic authentication.

11.3 Reliable Message Transport — wholly TBD

Reliable transport is intentionally undefined. This document defines no reliable header, sequence space, epoch, acknowledgment format, retry algorithm, flow control, timing, connection identity, restart behavior, or wire profile.

Older draft field sketches are not a contract and shall not be implemented as though they were one. Reliable transport shall be designed from concrete service requirements and published as a separate named profile with state-machine and conformance tests.

Until then, services may implement application-specific request/retry behavior over Unreliable Datagram, but shall not label it the standardized Reliable Message Transport.

12. Integrity and security distinctions

The following claims are separate:

- **Hardware error detection** concerns one physical/link transfer.
- **LLL integrity** concerns one encoded or reassembled LLL PDU.
- **E2E integrity** concerns accidental corruption or sequencing across the configured service path.
- **Cryptographic authentication** establishes an authorized origin and integrity against an attacker.
- **Confidentiality** prevents unauthorized reading.
- **Replay protection** rejects previously valid authenticated messages under defined freshness rules.

A CRC provides error detection, not authentication, confidentiality, authorization, or cryptographic replay protection. The core stack makes no default cryptographic-security claim. Secure carriers or explicitly designed secure services may be composed at an appropriate trust boundary.

13. Non-normative implementation guidance: SPSC pools

One suitable implementation uses statically allocated packet storage, one owner per global pool/free list, and bounded single-producer/single-consumer queues for cross-context ownership transfer.

This can provide deterministic memory use and zero-copy transfer, but it is **not** part of the LLL API or wire contract. Fixed ring SPSC queues, linked pooled SPSC queues, copied mailboxes, or other verified bounded mechanisms are all permitted.

If pooled linked SPSC queues are used:

- each queue has exactly one producer and one consumer;
- the pool/free list has one owner;
- handles have one owner at a time;
- publication is the synchronization boundary;

- pool and queue exhaustion have explicit behavior.

The initial/reference implementation should favor the simplest mechanism that can be verified on the target.

14. Open memory-model and coherency requirements

The generic architecture is not implementable safely across every target until the following platform contracts are made explicit:

- minimum atomic widths, alignment, and lock-free assumptions;
- release/acquire operations and required interrupt/core barriers;
- cache coherence assumptions for shared-memory queues and snapshots;
- DMA clean/invalidate ownership transitions and descriptor ordering;
- whether shared packet arenas may be cached;
- allocator lifetime and reclamation across LE restart;
- handle generation/ABA protection when compact indices are reused;
- behavior when a producer or consumer resets with records in flight;
- permitted borrowing across asynchronous DMA and execution-context boundaries;
- MPU/MMU permissions and isolation for shared storage;
- required behavior on non-coherent multicore systems;
- whether diagnostic counters are atomic, serialized, or snapshot-copied.

These are explicit gaps, not implementation details that may be assumed away. Each platform adapter shall document and test its choices. Cross-core and DMA-backed profiles cannot be promoted to stable implementation status until their applicable items are resolved.

15. Generic conformance expectations

An implementation of these contracts should be tested for:

- every TX acceptance and rejection result;
- ownership retention on rejection and release after acceptance;
- RX truncation, oversize, malformed, and integrity failures;
- queue/pool exhaustion without unbounded work or corruption;
- parser progress under arbitrary chunking;
- ISR-to-task and cross-core publication ordering;
- restart with Hardware Link and LLL work in flight;
- survival and eventual release of application-owned RX buffers across restart;
- same-core and cross-core `kLocalHost` semantic equivalence;
- snapshot replacement versus queue overflow behavior;
- static rejection of incompatible profile, capacity, and service combinations.

Wire-profile conformance is additional to these generic tests. System-level link, transport, congestion, restart, and archetype coverage belongs in [Prototype and Validation](#).

Link Entity Runtime and Status

Status and authority

Status: Provisional runtime design for prototype validation; normative only for prototype experiments; no runtime API or status wire service is stable.

Authority: Within that provisional scope, this document is authoritative for Link Entity ownership, execution, recovery, state lifetime, status semantics, bounded publication behavior, and validation expectations.

This document is the concise runtime companion to [Logical Links and Transports](#). It defines ownership, execution, recovery, and status responsibilities without defining a wire-status schema. The device and network context is summarized in [Architecture Overview](#).

Virtual-circuit identity, routing, and forwarding belong to [Core Protocol and Routing](#). Device status is an application service using those facilities; it does not alter link or virtual-circuit semantics.

Normative terms such as **shall** and **must** state prototype semantic requirements. They do not stabilize enum names, function signatures, object layouts, endpoint allocations, schemas, wire encodings, or any other public API.

1. Link Entity ownership

A device contains **1..N Link Entities (LEs)**. Each LE owns **1..N local runtime link instances**, where one runtime link consists of the relevant Logical Link and Hardware Link/driver state.

An LE owns:

- serialization of mutable LLL and link-driver state;
- parser/reassembly, TX scheduling, timers, and lifecycle state for its links;
- bounded ingress, egress, completion, and control ports;
- live status publication for state it can still observe;
- restart and cancellation of its transient runtime state.

An LE may run in a super-loop, task, process/executor, or dedicated core. Each LLL still has one serialized mutable execution context as required by [Logical Links and Transports](#).

An LE owns runtime links; it does not own a whole virtual circuit.

A virtual circuit may use one link, cross several links/LEs, or pass through other devices. Its identity and static route remain core configuration. Restarting an LE changes availability of its local link instances; it does not redefine, renumber, or take ownership of the virtual circuit.

A small device should normally use one LE unless scheduling, multicore placement, fault containment, or independent restart justifies more. Cross-LE forwarding uses explicit bounded ownership transfer and may copy when safe zero-copy transfer is unavailable.

2. Lifecycle contract

Links and LEs have independent lifecycle state.

A link shall support operations equivalent to:

```
disabled -> starting -> up
  ^           |           |
  |           v           v
+----- stopping <- degraded/faulted
```

An LE shall support operations equivalent to:

```
stopped -> starting -> running -> stopping -> stopped
                |
                v
            faulted -> restarting -> starting
```

Exact enum names are implementation-defined, but the contract shall provide:

- explicit start, stop, reset/reinitialize, enable, and disable requests;
- idempotent handling or explicit rejection of redundant requests;
- bounded request acceptance and completion reporting;
- a defined result for TX/RX work interrupted by stop or restart;
- rejection of new work while the relevant link/LE cannot accept it;
- a monotonically changing runtime generation or equivalent way to distinguish observations from before and after restart;
- ordinary handling of temporary link unavailability by routing and higher layers.

Normal lifecycle control also supports power management, maintenance, and configured reinitialization. It is not only a fault path.

Stopping or restarting shall not wait indefinitely for a failed driver or peer. The implementation shall define bounded quiesce and forced-cancellation behavior.

3. Incremental recovery

Recovery escalates from the smallest affected scope:

1. clear/restart the individual Hardware Link driver;
2. disable and re-enable the affected runtime link;
3. reconstruct or reset the owning LE's transient runtime;
4. restart a larger communications subsystem, if the platform has one;
5. reset the MCU/device only after lower-level recovery fails or system policy requires it.

The escalation policy shall define attempt limits, deadlines/backoff, and the conditions that move to the next level. Repeated automatic recovery shall not become an unbounded busy loop or diagnostic storm.

Uncertain transient work should be discarded rather than guessed back into a valid state. End-to-end transports or application services are responsible for recovering conversations that require continuity.

Recovery of one link should not stop healthy sibling links unless shared hardware, clocks, memory, or driver dependencies make isolation impossible. Such dependencies shall be represented in configuration and tests.

4. External supervision

An LE cannot be trusted to recover itself from every failure. A supervisor outside the monitored LE shall be able to observe at least:

- missing execution heartbeat or deadline;
- persistent lack of RX/TX progress where progress is expected;
- driver fault or repeated restart;
- invariant violation;
- sustained queue/buffer exhaustion;
- explicit LE fault transition.

The supervisor applies configured recovery escalation and records the reason/result outside disposable LE state.

The supervising path should depend on less infrastructure than the LE it supervises. A heartbeat updated by the failed task but consumed only by that same task is not external supervision.

Supervision is policy-driven: lack of traffic alone is not necessarily a fault on an idle link. Expectations, timeouts, and restart authority remain explicit configuration.

5. Independent debug path

Where hardware permits, provide a low-complexity debug path independent of the normal network runtime. It should avoid dependencies on:

- LLL parsing and routing;
- normal service dispatch;
- the primary packet-buffer pool;
- aggregate status publication;
- recoverable LE task state.

The path may be a dedicated debug interface, debugger channel, retained crash record, or another platform-specific mechanism. It is intended for boot progress, fatal fault evidence, and recovery observation, not routine high-volume traffic.

Debug access is privileged. Independence from the network stack does not exempt it from product security, production-disable, physical-access, or rate-limiting policy.

6. Transient, long-lived, and application-owned state

6.1 Disposable transient state

An LE restart may discard:

- partial frames and decoder state;
- incomplete profile-specific reassembly;
- pending timers and transient scheduler state;
- in-progress low-level transfers that cannot be recovered safely;
- queued work whose documented cancellation result is reported;
- live counters explicitly defined to reset with the runtime generation.

6.2 State that outlives an LE restart

The following shall live outside disposable LE runtime state:

- immutable/static configuration and named profile selection;
- persistent diagnostic counters required across LE restart;
- latched fault and restart records;
- supervisor state and recovery attempt history;
- resources intentionally shared by multiple LEs;
- storage backing buffers already transferred to application ownership.

Here, **persistent** means persistent across LE reconstruction. It does not necessarily mean nonvolatile across MCU reset or power loss. Each field shall have an explicit reset boundary.

6.3 Application-owned buffer survival

An RX buffer transferred to application ownership shall remain valid until the application releases or transfers it, even if the originating link or LE restarts.

Therefore:

- application-owned buffers cannot be allocated from storage destroyed with the LE runtime;
- restart shall not reclaim them by assuming all old-generation handles are free;
- release after restart shall remain safe;
- stale handles shall be detectable if compact slots can be reused;
- a restart may temporarily reduce available capacity while applications retain buffers, and that condition shall be bounded and observable.

An implementation may avoid this complexity by copying into application-owned storage before transfer, but it shall not invalidate an accepted ownership contract.

7. Live and latched status

Status has two distinct lifetimes.

Live status describes the current runtime generation:

- LE and link lifecycle state;
- enabled/configured/operational flags;
- heartbeat and last progress time;
- current queue/pool use and congestion state;
- current Hardware Link observations;
- counters explicitly scoped to the current generation.

Latched status preserves diagnostic evidence:

- last fault code, source, and timestamp;
- first/most severe fault where useful;
- LE and link restart counts;
- last recovery reason, action, and result;
- total bounded error/drop counters intended to survive LE restart;
- suppressed or rate-limited diagnostic count.

Live status may change rapidly and may become unavailable while an LE is faulted. Latched status remains readable from storage outside the failed runtime until its defined clear/reset boundary.

Snapshots shall include enough generation/sequence information to prevent consumers from combining unrelated fields from different updates.

8. Device-wide aggregate status service

Each device exposes **one aggregate Link Entity Status service**, not one independent network service per LE. It reports every local LE and each LE-owned runtime link.

The aggregate service:

- reads coherent live snapshots where available;
- reads latched records that survive failed-runtime reconstruction;
- identifies LE/link instances using stable configuration identifiers;
- distinguishes configured-disabled, starting, healthy, degraded, faulted, and unavailable observations;
- includes status age and runtime generation where needed;
- does not infer that a silent failed LE is healthy merely because its live snapshot stopped changing.

The service schema and endpoint allocation remain open. The application/service conventions are defined in [Application Protocols and Services](#).

9. Publication over healthy paths

Aggregate status shall be publishable over every currently healthy, configured path authorized to carry it. A failed link is not required to report its own failure.

For example, a healthy link owned by one LE may publish the latched fault and unavailable live state of another LE or link. Publication configuration remains static and subject to normal routing, capacity, priority, and security policy.

Requirements:

- status publication shall not depend exclusively on the failed path;
- no path is used unless configured and healthy enough to accept the PDU;
- failure to publish remotely does not erase local live/latched truth;
- each status-transmission outcome shall update its applicable local bounded counters, but shall not by itself trigger or schedule status publication;
- publication shall be driven only by the configured cadence or a valid explicit status request;
- each destination shall have at most one coalesced pending summary; a newer due summary updates that pending summary rather than creating another queued summary;
- a malformed status request or response, or any status publish failure, shall be recorded locally and shall never trigger a remote error, immediate status response, or another status publication;
- repeated faults are aggregated and rate-limited;
- diagnostic responses do not recursively generate diagnostic responses;
- status traffic remains bounded and shall not destabilize healthy control traffic.

A low-rate cadence-driven summary plus explicitly requested bounded detail is the required prototype pattern. The exact cadence, priority, destinations, and detail pagination are service-schema decisions.

10. Bounded diagnostics

Diagnostic storage and processing shall have fixed bounds. Implementations shall define:

- counter widths and saturating/wrapping behavior;
- maximum retained fault records;
- record replacement policy;
- maximum detail-response size and pagination/cursor behavior;
- publication rate and burst limits;
- suppression and dropped-diagnostic counters;
- clear authorization and reset boundary.

Per-byte, per-frame, or repeated identical fault logging shall not consume unbounded memory, CPU, or bandwidth. Local counters are the source of truth; remote diagnostics are a bounded projection.

11. Prototype restart and fault tests

11.1 Core prototype required tests

Every prototype shall validate:

1. Individual link stop/start, driver-fault detection, and successful link-only recovery.
2. Failed link recovery escalating through bounded quiesce and local LE reconstruction.
3. Healthy sibling links continuing through another link's recovery when the prototype configuration contains independently recoverable siblings.
4. A hung LE being detected by an external supervisor.

5. Partial RX, partial TX, pending timers, and queued work at each local restart boundary, including an explicit cancellation/completion result for every affected accepted TX.
6. Application-owned RX buffers surviving LE reconstruction and being released afterward, including bounded buffer/pool exhaustion caused by retained old-generation buffers.
7. Bounded ingress, egress, completion, and control queues reaching capacity and racing with local restart without corruption, ownership loss, or unbounded work.
8. Live snapshot generation changing without torn mixed-generation status.
9. Latched fault/restart evidence surviving LE reconstruction and obeying its defined clear/reset boundary.
10. Faulted-path status using another healthy configured path when one exists, and aggregate-service behavior when one or several LEs are silent.
11. Cadence- and request-driven status publication, one coalesced pending summary per destination, local transmission-outcome counters, and absence of diagnostic feedback publication.
12. Repeated fault injection with bounded records, rates, and suppression counts.
13. Configured-disabled links not being misreported or repeatedly recovered as faults.

11.2 Conditional platform tests

The following tests are required only when the prototype platform supports or selects the corresponding capability:

1. Independent debug-path operation while the normal LLL, router, pool, or status publisher is unavailable.
2. Restart races with DMA ownership, completion, cancellation, descriptor ordering, and required cache maintenance.
3. Recovery escalation from LE restart to MCU/device reset under its configured attempt limits and policy.
4. Cross-core queue and snapshot restart behavior on a non-coherent multicore platform, including required clean/invalidate and release/acquire operations.

A prototype that claims one of these capabilities shall run its conditional tests; an unselected or unsupported capability shall be recorded as not applicable rather than treated as missing core evidence.

12. Open schema and API issues

The following remain unresolved:

- exact LE and link lifecycle enums and transition/result APIs;
- ownership and timeout rules for asynchronous lifecycle commands;
- heartbeat/progress definition and external-supervisor interface;
- dependency groups for links that cannot restart independently;
- accepted-TX cancellation and completion result vocabulary;
- runtime generation representation and wrap behavior;
- persistent counter/record storage location and reset boundaries;
- coherent snapshot mechanism across tasks/cores;
- aggregate service endpoint ID, schema version, message sizes, and pagination;
- stable LE/link identifier allocation and representation;
- timestamps, clock domain, validity, and behavior before time synchronization;

- common versus profile-specific status fields;
- severity, fault-code registry, and extensibility rules;
- authorization for clear, restart, disable, and detailed-debug operations;
- exact healthy-path selection and duplicate publication policy;
- behavior when the aggregate service's own execution context fails;
- restart interaction with cross-core queues, non-coherent caches, and DMA;
- application-owned handle generation and allocator reclamation after restart.

These issues block stabilization of the runtime API or status wire service, but they do not change the ownership and recovery principles above. System-level fault-injection and promotion evidence belongs in [Prototype and Validation](#).

HDLC Logical Link Profile

EXPERIMENTAL — NOT A STABLE WIRE PROFILE

This document records useful framing candidates for prototyping. It does not define HDLC-v1, does not claim interoperability, and shall not be used as a released compatibility promise. Every candidate below remains subject to incompatible change until the stabilization criteria are met.

1. Scope

This candidate maps the canonical bounded-PDU service from [Logical Links and Transports](#) onto an ordered byte-stream Hardware Link such as a serial stream or FIFO. The byte stream may still suffer corruption, loss, overrun, or restart; this profile does not assume reliable delivery.

It provides candidate:

- frame delimiting and byte escaping;
- link-level error detection;
- bounded receive parsing and resynchronization;
- carriage of one serialized canonical PDU per frame.

It does not define application services, endpoint allocation, transport reliability, medium access, serial-port parameters, or Hardware Link electrical behavior. Canonical descriptor and routing semantics belong to [Core Protocol and Routing](#). Runtime ownership and restart behavior belong to [Link Entity Runtime and Status](#).

2. Candidate decoded-frame model

After delimiter removal and unescaping, the candidate decoded frame is:

```
+-----+-----+
| Leading link CRC-16 | Serialized canonical PDU |
| 2 bytes           | bounded, exact layout unresolved |
+-----+-----+
<----- CRC coverage ----->
```

Candidate properties:

- One frame carries exactly one complete canonical PDU.
- The CRC field is first.
- The CRC field is serialized least-significant byte first.
- All other multi-byte fields introduced by this profile would use little-endian serialization.
- The candidate CRC covers every decoded byte after the two CRC bytes and no delimiter or inserted escape byte.
- The receiver validates length and CRC before parsing any canonical PDU field.

The exact serialized canonical PDU layout is unresolved. Consequently, the minimum legal frame length, field offsets after the CRC, reserved-bit rules, and final profile vectors are also unresolved.

3. Candidate framing and escaping

3.1 Delimiters

`0x7E` is the candidate frame delimiter.

A transmitter emits a delimiter before and after the escaped frame body:

```
0x7E | escaped(decoded frame) | 0x7E
```

A receiver uses delimiters as resynchronization points. The likely parser model treats a delimiter as both the end of the preceding candidate frame and synchronization for the next frame, permitting adjacent frames to share a delimiter. This behavior must be frozen and covered by vectors before stabilization.

3.2 Escaping

Within the decoded frame, each occurrence of either:

- `0x7E`, or
- `0x7D`

is replaced on the byte stream by:

```
0x7D, original_byte XOR 0x20
```

Thus the canonical substitutions are:

```
0x7E -> 0x7D 0x5E
0x7D -> 0x7D 0x5D
```

The receiver reverses the XOR transformation before length accounting and CRC calculation. Delimiters and inserted escape bytes are not part of the decoded frame and are not covered by the CRC.

Whether a receiver must reject non-canonical escape pairs such as `0x7D 0x00`, rather than decoding them by the generic XOR rule, is unresolved. A stable profile must choose one behavior.

4. Candidate bounds

The decoded-frame limit candidate is:

```
MAX_DECODED_FRAME_SIZE = 4095 bytes, including the 2-byte CRC
```

This retains the earlier design candidate that limits the CRC-covered body to at most 4093 bytes. The claimed error-detection rationale for that limit must be independently verified for the final CRC profile and protected

message set before stabilization.

Derived candidate bounds:

- CRC-covered canonical PDU body: at most 4093 bytes.
- Standalone frame in the worst escaping case: at most $2 + 2 * 4095 = 8192$ stream bytes, including two delimiters.
- A streaming receiver need not buffer the escaped representation.
- The minimum legal decoded-frame size is TBD because the canonical PDU serialization is not frozen. It must be greater than two bytes.

An implementation may support a smaller local PDU limit for constrained targets only if deployment configuration proves peer/service compatibility. It shall still reject oversize input safely and expose its actual bound as a capability.

The stable profile must decide whether 4095 is a universal required receive limit, only a protocol maximum, or a profile-family maximum with named smaller-capacity variants.

5. Candidate CRC-16

The retained algorithm candidate is **CRC-16/CCITT-FALSE**:

```
width    = 16
poly     = 0x1021
init     = 0xFFFF
refin    = false
refout   = false
xorout   = 0x0000
check("123456789") = 0x29B1
```

Candidate frame processing:

1. Unescape bytes into the decoded frame while enforcing the decoded-size bound.
2. Require at least the final minimum decoded-frame length.
3. Read the leading CRC as a little-endian 16-bit value.
4. Compute CRC-16/CCITT-FALSE over `decoded_frame[2..end]`.
5. Compare the computed and transmitted values.
6. On mismatch, discard the frame and record a local integrity failure.
7. Only after a match, parse and validate the serialized canonical PDU.

These algorithm-level checks are useful during development but are not sufficient profile conformance vectors:

Protected bytes: <empty>

Computed CRC: 0xFFFF

Wire CRC bytes: FF FF

Protected bytes: 31 32 33 34 35 36 37 38 39 ("123456789")

Computed CRC: 0x29B1

Wire CRC bytes: B1 29

An empty protected body is not asserted to be a valid protocol frame; it is only an algorithm test.

Before stabilization, exact vectors shall include complete valid canonical PDUs, escaped CRC and payload bytes, minimum and maximum legal lengths, corrupted fields, and independently calculated expected outcomes.

6. No initial flow control

The initial experimental frame contains no flow-control field, credit nibble, receive-window advertisement, or link-control byte.

Local bounded TX rejection and backpressure follow [Logical Links and Transports](#). They do not provide peer-to-peer flow control. A service that can overrun a peer must currently constrain traffic through static provisioning or its own explicitly defined protocol behavior.

Any future peer flow control requires a separately specified extension or new named profile. It shall not be inserted into reserved bits or inferred from implementation behavior.

7. Candidate parser state

A bounded streaming parser likely needs only:

- synchronized/awaiting-frame state;
- accumulating-frame state;
- escape-pending state;
- discard-until-delimiter state after oversize or Hardware Link loss;
- decoded byte count;
- incremental CRC state or bounded frame storage;
- bounded diagnostics.

The parser shall make progress for every input byte and shall not allocate memory based on untrusted lengths.

CRC validation must precede canonical PDU parsing even when an implementation computes the CRC incrementally. If zero-copy receive storage is used, no unvalidated field may select a destination, extension length, endpoint, transport parser, or application handler before CRC success.

8. Parser edge cases requiring explicit behavior

The following cases shall have frozen behavior and conformance coverage before the profile can stabilize. Candidate handling is listed where the drafts provide a safe direction.

8.1 Synchronization and delimiters

- **Bytes before the first delimiter:** discard as unsynchronized input; optionally count one synchronization-loss event rather than one event per byte.
- **Repeated delimiters:** remain synchronized; do not deliver zero-length frames.
- **Opening delimiter immediately followed by a closing delimiter:** treat as idle/empty framing, not as a PDU.
- **A delimiter after accumulated bytes:** close the candidate frame and begin validation.
- **One delimiter between back-to-back frames:** likely closes one frame and synchronizes the next; this must be made normative.
- **Stream starts or restarts in the middle of a frame:** discard until the next delimiter.

8.2 Escapes

- **Escape split across Hardware Link read chunks:** preserve escape-pending state across calls.
- `0x7D 0x5E` and `0x7D 0x5D`: decode to `0x7E` and `0x7D`.
- **Escape immediately followed by delimiter:** discard the incomplete frame, count a malformed/dangling-escape event, and use the delimiter to resynchronize.
- **Escape at stream shutdown or driver reset:** discard transient frame state.
- **Non-canonical escaped value:** strict rejection versus generic XOR decoding is unresolved.
- **Unescaped `0x7D`:** it always begins an escape and cannot be emitted as decoded data by itself.

8.3 Length and storage

- **Fewer than the final minimum decoded bytes:** discard as undersize without parsing.
- **Exactly the configured maximum:** accept for CRC/PDU validation.
- **One byte beyond the configured maximum:** enter discard-until-delimiter state immediately; do not continue writing storage.
- **Long oversize run with escape bytes:** remain bounded and discard until an actual unescaped delimiter.
- **Worst-case escaping:** enforce decoded length, not the count of stream bytes.
- **Local implementation bound below the protocol candidate maximum:** reject above the advertised/configured local bound; deployment compatibility remains required.

8.4 Integrity and PDU validation

- **CRC mismatch:** discard locally before PDU parsing; do not generate a per-frame peer error.
- **Valid CRC but malformed canonical PDU:** record a PDU-parse diagnostic and discard or report through the bounded local diagnostics policy.
- **Valid CRC and unsupported canonical feature/profile:** follow the core protocol's bounded unsupported-feature behavior; do not let diagnostics recurse.
- **CRC bytes themselves require escaping:** escape them exactly like every other decoded byte.
- **CRC field byte order:** read and write little-endian.

8.5 Hardware Link failures

- **RX overrun, dropped DMA region, or explicit byte-loss indication:** abandon the current frame and discard until a delimiter.
- **Driver stop/restart:** clear parser transient state.
- **Read chunk contains several complete frames:** deliver each complete valid PDU in order until an upward queue becomes full; the policy for remaining bytes must be explicit and bounded.
- **Upward receive port is full after a valid frame:** drop, pause Hardware Link draining, or retain one bounded completed frame according to configured policy; count the event. Never overwrite unrelated storage silently.

9. TX behavior

TX submission follows the stable LLL acceptance and ownership contract.

For an accepted PDU, the experimental encoder:

1. validates the configured decoded-frame bound;
2. serializes the canonical PDU using the still-unresolved canonical layout;
3. computes the candidate CRC over that serialized PDU;
4. emits the little-endian CRC first;
5. escapes the CRC and PDU bytes;
6. emits delimiters according to the final framing rule.

The encoder shall either reserve enough bounded capacity before acceptance or retain sufficient owned state to resume partial Hardware Link writes. It shall not report acceptance and then depend on caller-owned bytes after their documented lifetime.

Partial byte-stream progress is local implementation state. It does not split the canonical PDU at the LLL service boundary.

10. Bounded diagnostics

At minimum, an implementation should maintain bounded counters or latched observations for:

- frames accepted;
- unsynchronized input/synchronization loss;
- empty delimiter runs, if diagnostically useful;
- dangling or invalid escape;
- undersize frame;
- oversize frame;
- CRC mismatch;
- canonical PDU parse/validation failure after valid CRC;
- RX storage or delivery-port exhaustion;
- Hardware Link overrun/loss indication;
- TX rejection and TX cancellation on restart;

- parser/driver restart count.

Diagnostics shall not allocate per-error records without a fixed bound, emit one log for every bad byte indefinitely, or generate recursive network errors. Repeated failures should be counted, aggregated, and rate-limited for remote publication. Counter width and saturation/wrap behavior remain to be specified.

11. Explicit unresolved wire and API issues

This experimental profile is blocked on:

1. Exact canonical PDU serialization, including descriptor layout, reserved bits, transport placement, and PDU minimum length.
2. Final profile name and version; HDLC-v1 is intentionally not assigned.
3. Confirmation or revision of the 4095-byte decoded limit and minimum required implementation capacity.
4. Independent validation of CRC-16/CCITT-FALSE suitability and the length/error-detection claim.
5. A complete set of exact full-frame CRC, escaping, boundary, and invalid-input vectors.
6. Strict versus permissive handling of non-canonical escape pairs.
7. Normative repeated/shared-delimiter behavior.
8. Exact behavior when upward delivery blocks while more stream bytes are already available.
9. Hardware Link error signaling required to detect byte loss and force resynchronization.
10. TX completion semantics and timestamp meaning for byte-stream drivers.
11. Diagnostic counter widths, saturation, reset, and publication schema.
12. Whether named smaller-capacity variants are interoperable profile variants or deployment constraints.
13. Serial parameters and any multidrop/half-duplex medium-access profile, if required; they are not part of this base candidate.

12. Conformance and stabilization criteria

This document may be promoted only after:

- the canonical PDU layout it carries is stable;
- a named, immutable profile and version are assigned;
- every wire field, byte order, bound, reserved value, and malformed-input behavior is normative;
- exact encoder/decoder vectors cover ordinary, escaped, boundary, and invalid frames;
- CRC vectors are independently reproduced by at least two implementations;
- streaming parsers pass arbitrary input chunking, fuzzing, and long-noise/oversize tests with bounded memory and time;
- C++, host-tool, and at least one embedded implementation interoperate using the same vectors;
- restart, byte-loss, RX overrun, delivery congestion, and partial-TX tests pass;
- capability/configuration checks reject incompatible local bounds;
- diagnostics are bounded and non-recursive;
- an implementer can build a parser and encoder without making an interoperability decision not stated in the profile.

Until every criterion is satisfied and reviewed, this remains an experimental prototype profile. Broader integration and promotion evidence belongs in [Prototype and Validation](#).

Classical CAN PDU Adapter

Status: Experimental / Work in Progress

Scope: Logical Link Layer adaptation of bounded PDUs to Classical CAN

Interoperability: Not yet claimed; no wire profile is frozen

This document separates three kinds of material:

- **Accepted contract** — architectural requirements that implementations shall preserve.
- **Candidate design** — the current prototype direction, explicitly subject to incompatible change.
- **Unresolved items** — decisions that must not be inferred from old drafts or candidate examples.

Normative words such as **shall** and **must** apply only to the accepted contract unless a named wire profile later adopts a candidate rule. Candidate layouts are explanatory byte layouts, not C or C++ ABI definitions.

1. Relationship to the Consolidated Architecture

Read this specification with:

- the [architecture overview](#) for scope and system boundaries;
- [core protocol and routing](#) for endpoint, Virtual Circuit, direction, listener, forwarding, and replication semantics;
- [application protocols and services](#) for publication and transport use;
- [logical links and transports](#) for the generic LLL contract and backpressure;
- [prototype and validation](#) for system-level validation;
- [rationale, use cases, and risks](#) for non-normative context.

The [document index](#) and [authority guide](#) controls the relationship between this specification and the source drafts.

2. Accepted Adapter Contract

2.1 Service

The CAN PDU Adapter is a Classical CAN Logical Link Layer. It carries one complete, bounded upper-layer PDU in N constituent CAN data frames.

The adapter shall provide:

1. **Unreliable datagram delivery.** It does not guarantee that an accepted PDU reaches a receiver.
2. **All-or-nothing receive visibility.** A complete validated PDU is delivered upward, or nothing is delivered.
3. **No adapter-level PDU ACK or retry.** Reliability, acknowledgment, duplicate suppression, and end-to-end retransmission belong to a higher transport when required.
4. **Bounded operation.** Maximum PDU size, frame count, active contexts, queueing, and reassembly memory are statically bounded.

5. **Allocation-free feasibility.** A conforming implementation can use fixed buffers and fixed context tables without runtime allocation.

Classical CAN controller acknowledgment and automatic retransmission remain below this adapter. They do not constitute a PDU acknowledgment and do not change the adapter's unreliable service contract.

2.2 Framing and ownership invariants

- Every constituent frame of one PDU shall use the same CAN arbitration ID.
- A protocol CAN ID shall have one statically unique transmitting owner in every state where it can be used. Two nodes must not transmit different data under the same ID.
- Raw CAN and protocol-adapter traffic shall occupy disjoint, statically configured CAN-ID domains.
- Raw CAN traffic shall retain raw-link semantics and incur no PDU-adapter data-field overhead.
- Small **N** is preferred because latency, bus occupancy, and probability of whole-PDU loss increase with aggregation depth.
- No upper-layer callback, queue, endpoint, listener, forwarder, or transport may observe a partial PDU.

Static configuration shall prevent CAN-ID collisions; an on-wire discriminator alone does not prove collision-free coexistence.

2.3 Topology boundary

This adapter does not provide general ECU-to-ECU unicast routing. In particular, it does not create an address by which ECU X can directly target arbitrary ECU Y across buses.

CAN and other supported links may have any number of passive listeners. An ECU may therefore consume another ECU's publication when higher-layer configuration permits it.

Two distinct higher-layer topology operations are permitted and shall not be conflated:

- **Transparent forwarding** carries a completed publication onward on the same Virtual Circuit while preserving its canonical routing and publication semantics.
- **Cross-VC replication** consumes a completed publication on one Virtual Circuit and reoriginates a publication on each statically configured destination Virtual Circuit, including state-sharing groups. Continuing the same VC across another physical bus segment is transparent forwarding, not replication.

Listening, transparent forwarding, and cross-VC replication are explicit pub/sub behaviors. None provides general ECU routing, and none operates on adapter fragments.

2.4 Diagnostics and interoperability

- Implementations shall maintain bounded local counters for accepted, rejected, completed, aborted, delivered, and discarded work.
- Malformed traffic shall be counted and dropped locally; it shall not cause an adapter-level error response.
- Exact counter names, widths, aggregation, and remote diagnostic schema remain unresolved.
- Independent interoperability shall not be claimed until an immutable named profile and its positive and negative conformance vectors are published.
- An incompatible wire change requires a new profile name/version. A published profile is never silently reinterpreted.

3. Accepted State-Machine Requirements

3.1 Transmit

Before emitting the first frame, TX shall:

1. validate the selected route/profile and CAN-ID ownership;
2. verify that the complete PDU fits configured bounds;
3. obtain all storage and queue capacity required by its documented submission contract;
4. preserve a stable view of the PDU bytes and metadata for the duration of frame emission.

TX shall then emit one finite frame sequence under one CAN ID. It shall not report remote delivery success. If emission aborts after the first frame, TX shall stop the attempt, release bounded local state, and increment a local abort reason; it shall not retry the PDU as adapter behavior.

An implementation may reject a local submission because of congestion or invalid configuration. Such rejection means the PDU did not enter the link and is distinct from loss after transmission begins.

3.2 Receive

RX shall keep incoming bytes private to the adapter until all required frames are present and all applicable structural, length, mapping, and integrity checks pass.

Each active reassembly context shall have statically bounded storage and lifetime. On any failure that makes completion ambiguous or invalid, RX shall discard the complete context. It shall never deliver the validated prefix of an incomplete or rejected PDU.

The final transition is atomic from the upper layer's perspective:

```
collecting -> complete and valid -> one whole-PDU delivery
collecting -> invalid/incomplete -> discard with no delivery
```

The exact context key, collision policy, timeout, and ordering rules are wire-profile decisions and are not accepted as frozen below.

4. Current Candidate Wire Design — Not Frozen

Everything in this section is a prototype candidate. It must not be used as an interoperability claim.

4.1 Candidate profile summary

- Classical CAN data frames with at most eight data bytes.
- `N = 1..16`; `N = 1..4` is recommended and larger values require explicit justification.
- One `SequenceControl` byte in every protocol frame.
- An explicit `START` indication, a message generation, and a four-bit frames-remaining countdown.
- Exact encoded length derived from each frame's DLC; no unused-byte trailer.
- No additional PDU CRC for `N = 1`.
- CRC-8 for `N = 2..3`; CRC-16 for `N >= 4`.

- The multi-frame PDU CRC appears in the START frame.
- A compact endpoint alias and VCN appear in the START frame.
- One active PDU per CAN ID and no same-ID interleaving as the safe prototype default.

4.2 Candidate SequenceControl

```
bit 7      START
bits 6:4    MessageGeneration[2:0]
bits 3:0    FramesRemaining
```

Candidate interpretation:

- The START frame has `START = 1` and `FramesRemaining = N - 1`.
- Every continuation has `START = 0`.
- `FramesRemaining` decrements by one on each constituent frame.
- The final frame has `FramesRemaining = 0`.
- A single-frame PDU has `START = 1` and `FramesRemaining = 0`.

The START frame also currently combines the upper generation bits with VCN:

```
VirtualCircuitAndGeneration:
bits 7:5    MessageGeneration[5:3]
bits 4:0    VCN
```

This yields a candidate six-bit generation. Continuations carry only the low three bits and would bind to state established by START. Generation width, advancement, wrap handling, and binding rules are unresolved.

4.3 Candidate compact START layouts

The compact candidate assumes three per-PDU metadata bytes: `ProtocolControl`, `VirtualCircuitAndGeneration`, and an 8-bit endpoint alias.

```
N = 1:
byte 0      SequenceControl
byte 1      ProtocolControl
byte 2      VirtualCircuitAndGeneration
byte 3      endpoint_alias
byte 4..    upper-layer PDU bytes
```

```
N = 2..3:
byte 0      SequenceControl
byte 1      CRC-8
byte 2      ProtocolControl
byte 3      VirtualCircuitAndGeneration
byte 4      endpoint_alias
byte 5..    upper-layer PDU bytes
```

```
N >= 4:
byte 0      SequenceControl
byte 1..2    CRC-16
byte 3      ProtocolControl
byte 4      VirtualCircuitAndGeneration
byte 5      endpoint_alias
byte 6..    upper-layer PDU bytes
```

```
Continuation:
byte 0      SequenceControl
byte 1..    upper-layer PDU bytes
```

The endpoint alias is link-local and would map statically to the canonical endpoint identity before delivery. Any optional group or multicast representation must use a separate explicit mechanism; `UpstreamHostAlias` cannot carry multicast or broadcast semantics. Exact control bits, CRC byte order, VCN semantics, endpoint-alias profiles, and the separate multicast representation remain unresolved.

4.4 Candidate DLC rule

Non-final multi-frame frames would normally use `DLC = 8`. The final frame, including an `N = 1` frame, would use the DLC for its exact encoded byte count. Bytes beyond DLC would not exist for reassembly or CRC coverage.

Zero-length PDU validity, minimum legal DLCs, short non-final frames, and all malformed-DLC behavior remain unresolved.

4.5 Candidate standard-ID mappings

The default 11-bit candidate reserves an enable/discriminator bit:

```
bits 10:9   CAN priority code
bits  8:4   Index
bit    3    Direction
bits  2:1   UpstreamHostAlias
bit    0    protocol enable

protocol enable = 0   raw/legacy CAN domain
protocol enable = 1   PDU-adapter domain
```

The two-bit `UpstreamHostAlias` has exactly four singular alias codes:

```
00 alias 0 -> profile-defined singular canonical UpstreamHost
01 alias 1 -> profile-defined singular canonical UpstreamHost
10 alias 2 -> profile-defined singular canonical UpstreamHost
11 alias 3 -> profile-defined singular canonical UpstreamHost
```

A future named profile must define a static one-to-one mapping from every alias code to exactly one singular canonical `UpstreamHost` value. None of the four codes has broadcast, multicast, wildcard, or other collective-host semantics.

An alternate expanded-host candidate uses:

```
bits 10:9   CAN priority code
bits  8:4   Index
bit    3    Direction
bits  2:0   UpstreamHostAlias
```

The three-bit `UpstreamHostAlias` has exactly eight singular alias codes:

```
000 alias 0 -> profile-defined singular canonical UpstreamHost
001 alias 1 -> profile-defined singular canonical UpstreamHost
010 alias 2 -> profile-defined singular canonical UpstreamHost
011 alias 3 -> profile-defined singular canonical UpstreamHost
100 alias 4 -> profile-defined singular canonical UpstreamHost
101 alias 5 -> profile-defined singular canonical UpstreamHost
110 alias 6 -> profile-defined singular canonical UpstreamHost
111 alias 7 -> profile-defined singular canonical UpstreamHost
```

The profile must define a static one-to-one mapping from every alias code to exactly one singular canonical `UpstreamHost` value. There is no implicit all-host code and no broadcast, multicast, or wildcard semantics. Canonical `UpstreamHost` values that are special, unrepresentable, or absent from the configured alias map shall be rejected. `UpstreamHost::kLocalHost` is invalid on external CAN and shall be rejected on transmit and receive.

`CAN priority code` is a link-local arbitration input, not an encoded canonical QoS value. A future named profile must provide explicit mapping vectors from supported symbolic canonical QoS values to CAN priority codes and the resulting CAN arbitration order. An implementation shall never copy canonical QoS bits into this field or infer numeric ordering among canonical QoS symbols.

The expanded mapping has no discriminator and therefore assumes the complete selected standard-ID space is assigned consistently to the protocol. Neither mapping is frozen. Canonical Direction, `UpstreamHost`, and `Index` mapping rules belong to the core routing contract and remain under reconciliation. These CAN-ID naming and semantic corrections do not change either candidate's bit allocation, layout, or capacity.

5. Candidate Capacity

For the compact candidate at maximum DLC, let:

```
C = 0 bytes for N = 1
C = 1 byte  for N = 2..3
C = 2 bytes for N >= 4
```

Every frame spends one byte on `SequenceControl`, and the PDU spends three bytes on compact metadata. Maximum upper-layer PDU capacity is therefore:

```
P_max(N) = 8N - N - 3 - C
           = 7N - 3 - C
```

Candidate maxima for `N = 1..16` are:

N:	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
P_max:	4	10	17	23	30	37	44	51	58	65	72	79	86	93	100	107 bytes

If an optional multicast extension consumes `M` bytes and the selected transport consumes `T` bytes, candidate application capacity becomes:

```
A_max(N) = 7N - 3 - C - M - T
```

These values exclude CAN arbitration, control, frame CRC, ACK, stuffing, and inter-frame overhead. They also assume the compact alias profile and the candidate CRC thresholds.

Reliable transport is not designed. Its overhead remains `T_reliable = TBD`; therefore this specification makes **no fixed reliable-capacity claim**.

6. Prototype-Safe Candidate State Machines

These state machines define a conservative prototype baseline, not a frozen wire contract.

6.1 TX candidate

IDLE

- > PREPARE
- > EMIT_START
- > EMIT_CONTINUATIONS
- > COMPLETE
- > IDLE

Any local failure after EMIT_START

- > ABORT
- > IDLE

PREPARE would select the smallest legal **N**, encode metadata, select CRC by **N**, compute the CRC when present, and assign the next generation. The sender would emit all frames under one CAN ID in countdown order and would not interleave another PDU under that ID.

Different CAN IDs may naturally interleave through CAN arbitration. Whether one adapter may schedule multiple local CAN IDs concurrently is an implementation resource decision.

6.2 RX candidate

The safe baseline keeps at most one active PDU per CAN ID:

IDLE + valid START

- > allocate/reset bounded context
- > COLLECTING

COLLECTING + expected continuation

- > append bytes
- > refresh bounded timeout

COLLECTING + expected final frame

- > validate length, mappings, structure, and CRC
- > DELIVER whole PDU or DISCARD
- > IDLE

COLLECTING + any ambiguous or invalid event

- > DISCARD complete context
- > IDLE

A new valid START under a CAN ID would discard and replace an incomplete context under that ID. An unexpected continuation, generation mismatch, or countdown mismatch would discard the affected context. This fail-closed behavior is suitable for a prototype, but duplicate, stale, out-of-order, collision, and replacement details must be frozen by the eventual profile.

A fixed global context pool may bound simultaneous reassembly across different CAN IDs. Pool exhaustion discards the newly arriving candidate PDU or follows another deterministic profile rule; it must never allocate dynamically or evict silently without a counter.

7. Discard and Abort Accounting

The exact diagnostics schema is unresolved, but prototype instrumentation should distinguish at least:

- invalid or unsupported CAN frame class;
- protocol ID outside configured ownership/domain;
- illegal DLC or encoded length;
- malformed START or illegal `N`;
- continuation without an active context;
- unexpected START/context collision;
- generation mismatch or stale frame;
- duplicate, skipped, or out-of-order countdown;
- reassembly timeout;
- context-pool or buffer exhaustion;
- PDU larger than configured maximum;
- CRC mismatch;
- invalid control/reserved bits;
- unknown endpoint alias or VCN mapping;
- unknown, duplicate, special, unrepresentable, or non-singular `UpstreamHostAlias` mapping;
- external-CAN use of `UpstreamHost::kLocalHost`;
- multicast, broadcast, or wildcard use through `UpstreamHostAlias`;
- missing or incompatible alias-map digest or fail-closed mapping configuration;
- missing or invalid symbolic QoS-to- `CAN priority code` mapping;
- invalid route, group, or broadcast encoding;
- upper-layer delivery rejection;
- TX validation, congestion, queue, controller, or mid-sequence abort.

Raw-domain frames are not malformed protocol frames; they are delivered to the configured raw path or ignored according to raw-link configuration.

8. Security and Non-Goals

The CAN frame CRC and candidate PDU CRC detect accidental corruption. They do not provide authentication, authorization, confidentiality, freshness, replay protection, or proof of source identity.

A node with bus access may inject, replay, delay, suppress, or flood frames. Static CAN-ID ownership is a configuration invariant, not a cryptographic control. Resource bounds, acceptance filters, timeouts, priority analysis, and rate-limited diagnostics reduce fault impact but do not make an untrusted bus secure.

This adapter does not provide:

- general ECU-to-ECU routing or internetworking;
- dynamic discovery, route negotiation, or profile negotiation;
- adapter-level reliable delivery;
- transport ordering or duplicate suppression above one reassembly attempt;
- end-to-end safety protection or security;
- arbitrary segmentation for large objects or byte streams;
- multicast membership or state-replication policy;
- remote diagnostic responses to malformed traffic.

Security-sensitive or remotely reachable deployments require reviewed higher-layer protection and gateway policy.

9. Unresolved Wire and Behavior Decisions

The following remain explicitly open:

1. Exact bit layouts, `ProtocolControl`, reserved-bit policy, and multicast extension.
2. Canonical routing reconstruction, Direction mapping, and named-profile symbolic QoS-to- `CAN priority code` mapping vectors and arbitration order.
3. The final standard-ID `UpstreamHostAlias / Index` profile, its static one-to-one singular canonical `UpstreamHost` map, ownership rules, rejection of special or unrepresentable hosts and external-CAN `UpstreamHost::kLocalHost`, alias-map digest, and fail-closed configuration behavior.
4. Standard versus extended identifiers and handling of RTR, error, and CAN FD frames.
5. Generation width, allocation, wrap, interleaving, and reassembly context keys.
6. Whether one active PDU per CAN ID becomes the mandatory safe profile.
7. Reassembly timeout definition, start point, refresh behavior, and configuration units.
8. Duplicate, out-of-order, skipped, stale, unexpected-START, and collision behavior.
9. CRC-8 and CRC-16 polynomial, initialization, reflection, xor-out, residue, coverage, byte order, and test vectors.
10. Whether CRC coverage includes CAN ID, generation, `N`, exact length, metadata, and all PDU bytes.
11. Zero-length PDUs, minimum DLCs, short non-final frames, and every DLC edge case.
12. Compact alias and literal endpoint profiles, mapping validation, and unknown-alias behavior.
13. Group and broadcast encoding, interaction with passive listeners, and proof that multicast, broadcast, and wildcard semantics cannot pass through `UpstreamHostAlias`.
14. Diagnostics counter schema, widths, saturation, reset, publication, and rate limiting.
15. Profile identifiers, configuration fingerprints, and how tools prove peer compatibility without dynamic negotiation.

No implementation should fill these gaps silently and call the result interoperable.

10. Prototype Validation Matrix

Area	Required prototype cases	Pass condition
Capacity boundaries	N=1,2,3,4,16 ; payloads immediately below, at, and above each maximum	Minimal legal N ; oversize rejected before START
Exact length	Every legal candidate final DLC; short and malformed DLCs	Exact reconstructed length or deterministic discard
Loss	Drop START, middle, and final frames	No partial delivery; timeout/counter is correct
Ordering	Duplicate, swap, skip, delay, and replay constituent frames	No false delivery; deterministic discard accounting
Generation	Rapid wrap, stale continuations, reset, and new START collision	No cross-generation assembly
Concurrency	Several CAN IDs interleaved; attempted same-ID interleaving	Bounded correct contexts; safe same-ID behavior
Integrity	Corrupt every protected field and payload position	CRC behavior matches golden vectors
Domains/IDs	Raw and protocol frames across reserved ID boundaries; duplicate-owner configuration	Correct dispatch; configuration collision rejected
Mapping	Valid/invalid VCN, endpoint-alias, and UpstreamHostAlias mappings; four-code and eight-code singular host maps; compact/literal experiments	Canonical metadata reconstructed only from a configured one-to-one singular mapping, otherwise PDU discarded
Mapping configuration	Matching, missing, stale, and incompatible alias-map digests; duplicate canonical targets; fail-closed startup and reconfiguration	Only the intended alias map becomes active; mismatch or ambiguity cannot enable traffic
Host markers/groups	External-CAN UpstreamHost::kLocalHost ; special and unrepresentable hosts; attempted multicast, broadcast, or wildcard through UpstreamHostAlias	Every case rejected; no group delivery is inferred from a host alias
QoS/arbitration	Named-profile vectors for every supported symbolic canonical QoS value and CAN priority code; attempts to copy or numerically order canonical QoS	Code and arbitration order match vectors exactly; implicit mapping is rejected
Frame classes	Standard, extended, RTR, and CAN FD inputs	Profile policy applied consistently
Resource bounds	Full context pool, queue exhaustion, timeout storms, malformed-frame fuzzing	No allocation, overrun, livelock, or unbounded diagnostics
Topology	Passive same-bus listeners, same-VC transparent forwarding, and configured cross-VC replication	Transparent forwarding preserves canonical semantics; replication consumes/reoriginates; neither implies ECU unicast
Implementation	At least two independent encoders/decoders consume the same vectors	Byte-identical output and matching accept/reject results

System validation shall also measure worst-case RAM, CPU time, queue occupancy, latency, bus load, and counter behavior under error storms.

11. Stabilization Exit Criteria

The adapter may advance from Experimental/WIP only when:

1. Every item in Section 9 is resolved or explicitly excluded by the named profile.
2. Frame classes, CAN-ID mapping, byte layouts, bit numbering, DLC rules, maximum `N`, timeout, and all state transitions are normative.
3. CRC algorithms and complete coverage are justified and published with check values and byte-exact vectors.
4. Valid and invalid conformance vectors cover all boundary and failure cases.
5. Static tooling detects duplicate CAN-ID ownership, domain overlap, illegal mappings, incompatible profiles, and impossible PDU/MTU configurations.
6. Fixed memory and timing bounds are demonstrated on representative small targets.
7. Two independent implementations interoperate under loss, corruption, wrap, restart, and malformed-input testing.
8. Diagnostics are bounded, locally useful, and incapable of recursive error traffic.
9. Cross-document terminology and routing semantics agree with the consolidated core and LLL contracts.
10. A security and fault-containment review documents the trusted-bus assumptions and denial-of-service limits.

Completion of Reliable Transport is not an exit criterion for the unreliable adapter. Any later reliable-capacity statement must use the separately specified `T_reliable` and its own conformance work.

12. Source Lineage

This specification consolidates the experimental material in:

- `can_pdu_adapter_reorganized.md`;
- `can_pdu_adapter.md`;
- `can_pdu_payload_capacity(1).md`;
- `can_pdu_payload_capacity.md`;
- the Classical CAN sections of `network_stack_design_updated.md`;
- relevant updates in `recent_protocol_design_updates.md` and `network_architecture_refinements.md`.

Those files remain historical source inputs. This document controls within its declared scope while preserving all unfinished wire decisions as unfinished.

Prototype and Validation

Status: Actionable prototype plan

Maturity: Disposable validation implementation; no stable API or wire-compatibility promise

Security posture: Trusted local/private development environment only

This prototype proves the smallest coherent embedded message fabric before production services or additional transports are frozen. It implements [application protocol and service](#) semantics over the addressing model in [core protocol and routing](#) and the contracts in [logical links and transports](#). The [README](#) and [architecture overview](#) define authority and scope.

The four application protocols below are disposable test instruments. Their message names, APIs, endpoint IDs, and serialized layouts are provisional. The prototype must record semantic requirements and generate repeatable vectors, but it must not present an expedient encoding as a final ecosystem profile.

1. Initial implementation boundary

The first end-to-end slice contains:

- canonical in-memory endpoint/message metadata;
- literal endpoint identity using the 2-bit namespace and IDs `0x0100..0xFFFF`;
- explicit upstream/downstream service roles;
- Unreliable Datagram only;
- Service Router registration, validation, dispatch, and bounded TX results;
- latest-value snapshot and bounded event-queue delivery;
- single-writer and multi-caller TX paths;
- static VC, endpoint, peer, listener, forwarding, and replication configuration;
- loopback/in-process and shared-memory local links for `VirtualCircuit::kLocalHost` with its `UpstreamHost::kLocalHost` companion marker;
- one external reference link profile/technology, with as many configured link instances/segments as the fixtures require;
- packet/message capture, counters, fault injection, and link/restart diagnostics.

The default external profile is a process-isolated virtual datagram link with multiple harness-controlled ports. Separate processes and configured virtual ports provide external-link boundaries, independent capture, and multiple link instances/segments without exposing an unauthenticated prototype carrier to a physical or routed network. "One external link" means one external link profile/technology, not one link object, socket, port, bus, or segment.

Non-loopback UDP is not the default. Gate 0 may approve it only as an exception on an isolated, non-routed test network with exact local-interface and peer allowlists, a host firewall, and NAT and IP forwarding disabled. Its profile, capture, and report must prominently state that the carrier is unauthenticated and must identify the approved interfaces and peers.

Reliable transport is deferred. Reliable, CAN, and HDLC completion are not initial dependencies or acceptance prerequisites. HDLC and CAN remain experimental and are added only after their parsers, integrity rules,

bounds, and negative vectors are ready enough not to obscure validation of the common architecture. The [CAN PDU adapter](#) is not an initial dependency, and reports retain these expected gaps rather than implying that the corresponding production paths were validated.

2. Disposable validation protocols

All four protocols use a provisional canonical schema and begin semantically with `protocol_version` and `message_type`. The deployment assigns nonfinal literal endpoint IDs; no alias is visible to service code. Each protocol declares bounded sizes, rates, storage, and TX-rejection behavior before implementation.

Producer attribution is expressed as the configured `ProducerKey` plus recorded provenance. It is not inferred from route metadata or represented by a generic network-source label. Gate 1 must define the prototype's working `ProducerKey` metadata, derivation, and provenance rules so tests are reproducible, but this does not reserve or freeze a source field in a future wire profile.

Protocol A — latest-state publication

Purpose: prove one-producer/many-listener publication, newest-value storage, freshness, and congestion coalescing.

Semantic messages:

- `StateSnapshot`: current bounded state, validity, producer sequence or epoch, and a defined time/age basis.

Behavior:

- one upstream publisher is authoritative;
- configured same-link and local listeners retain only the newest valid snapshot;
- missed intermediate values are acceptable and observable through sequence gaps;
- stale state becomes invalid after a configured age;
- one pending TX value is retained; a newer value replaces it under congestion;
- producer restart and sequence discontinuity are distinguishable from ordinary loss.

No acknowledgment or reliable delivery is implied.

Protocol B — command, feedback, and asynchronous event

Purpose: prove directed roles, request-scoped reply context, autonomous downstream transmission, and mixed storage policies.

Semantic messages:

- `Command`: bounded upstream intent with command correlation and freshness/precondition data;
- `CommandFeedback`: immediate acceptance or service-level rejection;
- `CurrentFeedback`: latest achieved state;
- `CompletionEvent`: bounded asynchronous downstream completion/failure event.

Behavior:

- immediate feedback uses the request-scoped reply context;

- periodic/current feedback uses a statically configured downstream-to-upstream path;
- completion events use static or explicitly learned peer binding, never an expired request context;
- request-scoped responses and autonomous streams/events use separate endpoint registrations; the prototype does not combine their binding modes in one registration;
- the downstream service remains in the downstream role while sending feedback or events to an endpoint in the upstream role; traffic direction does not reverse either endpoint's configured role;
- duplicate and stale commands have explicit service behavior even though the transport does not retry;
- current feedback uses a latest-value slot; completion events use a bounded FIFO;
- local TX rejection does not advance command acceptance or event state.

The prototype does not claim exactly-once execution.

Protocol C — minimal RPC

Purpose: prove bounded request/response dispatch, correlation, service errors, and timeout behavior without hiding reliability inside the application.

Semantic messages:

- `Request` : operation discriminator, correlation value, and bounded arguments;
- `SuccessResponse` : matching correlation and bounded result;
- `ErrorResponse` : matching correlation and compact service reason.

Behavior:

- one caller targets one configured service instance;
- the callee replies using request-scoped context;
- caller state has a fixed maximum number of outstanding requests;
- unknown operation, malformed arguments, busy state, and unsupported version/type are explicit;
- timeout ends the local attempt; the Unreliable Datagram prototype performs no automatic retry;
- any optional caller retry is a test policy and requires the chosen operation to be idempotent or carry an application transaction key.

Protocol D — DRIP-style fault/event reporting

Purpose: prove bounded diagnostics that remain observable without creating an error storm.

Semantic messages:

- `HealthSummary` : periodic counts, highest severity, last code, lost/suppressed counts, and restart state;
- `DetailRequest` : bounded range/cursor request;
- `DetailResponse` : bounded record batch plus continuation state;
- optional `FaultEvent` : sequenced best-effort event for low-latency observation.

Behavior:

- local counters and a bounded record ring are authoritative;
- summary publication continues at a configured modest rate;
- detail is pulled explicitly, pauses on congestion, and resumes from bounded cursor state;

- repeated faults may aggregate; suppression and record loss remain visible;
- diagnostic failures and malformed requests never generate recursive remote errors;
- diagnostic traffic cannot block or outrank the state/command traffic used by the fixture.

3. Deployment fixtures

Each fixture is generated from the executable static topology/configuration schema established at Gate 0 and has a capture point at every link boundary. Before an acceptance run, its manifest fixes the enabled capabilities and bindings, queue depths, overflow policy, TX-result vocabulary, multi-caller ordering, clock/test values, metric registry, capture schema, counter semantics, expected counter deltas, fault seeds/tolerances, and resource budgets required by Gates 3 and 4.

F1 — same-core `VirtualCircuit::kLocalHost`

Run all four protocol pairs in one process or one MCU core. Messages traverse the router and loopback LLL rather than calling service implementations directly. This proves API consistency, dispatch, storage policies, and deterministic host tests. `VirtualCircuit::kLocalHost` selects the local domain and carries the required `UpstreamHost::kLocalHost` companion marker; the core protocol owns their representation and encoding details.

F2 — cross-core `VirtualCircuit::kLocalHost`

Place producer/client and consumer/server on different cores or core-simulating execution contexts connected by bounded shared memory. The paired local VC and host marker never reach an external LLL. Exercise coherent and, where supported, explicit cache-maintenance configurations.

F3 — several Main hosts on `kMainCompute`

Use at least three Main instances. Exercise directed A/B/C/D service instances between selected Mains, passive observation where configured, and independent `UpstreamHost` identities. `kMainCompute` uses the normal endpoint/service model; it is not a separate inter-Main protocol.

F4 — same-bus listeners

Connect one Main and several ECU fixtures to one externally carried logical bus. An ECU publication retains its configured `ProducerKey` and producer-attribution provenance while the Main and other configured ECUs listen because the link is broadcast-capable. Listeners do not become addressed peers, send acknowledgments, or acquire reply context.

This fixture must demonstrate that the complete downstream target is `VCN + Index` and that `UpstreamHost` identifies a Main rather than serving as a general ECU address.

F5 — configured multi-bus state replication

Place members of one state-sharing group on at least two buses. A statically configured replicator consumes Protocol A state and originates bounded copies on the configured destination VCs. Its manifest declares the eligible Protocol A message types, source and destination `ProducerKey`s, authority owner, provenance transformation, and conflict policy. Validate loop-free fan-out, producer attribution, capacity accounting, conflict handling, and restart behavior. Negative fixtures reject replication of Protocol B commands, Protocol C

RPC, Protocol C/D errors, and any undeclared message type, and reject a competing producer for a single-authority destination.

This is an explicit application/deployment pattern. It does not provide arbitrary MCU-X-to-MCU-Y unicast, infer routes from endpoint IDs, or create general ECU routing across buses.

F6 — same-VC forwarding across segments

Connect two configured link instances/segments of the same external link profile and install only the declared same-VC forwarder between them. For every forwarded datagram, the canonical descriptor, payload bytes, `ProducerKey` /origin context, peer context, and transport state must remain unchanged end to end. Only hop-local LLL framing or envelope data may change. Captures on both segment boundaries prove byte/field equality and show that the forwarder performs no application decode/re-encode, producer rewrite, peer promotion, transport transition, or implicit route creation.

4. Prototype layering and components

Implement components in separable layers:

```
Disposable A/B/C/D protocol semantics and provisional schemas
|
Typed codecs and validation
|
Endpoint policies: callback, snapshot, event FIFO, TX funnel
|
Service Router: identity, role, peer context, dispatch, static VC policy
|
Unreliable Datagram
|
Loopback / shared-memory / external datagram LLLs
|
Platform driver, scheduler, clock, and memory adapters
```

Cross-cutting components are:

- generated or declarative static configuration;
- fixed-capacity packet/message storage;
- capture with canonical metadata and raw-link records;
- metrics, persistent restart/fault counters, and DRIP reporting;
- Link Entity aggregate status and bounded status delivery;
- deterministic fault-injection hooks;
- host reference codecs and conformance runner.

The common layers must not depend on UDP socket concepts, CAN arbitration fields, or shared-memory pointers. Link adapters reconstruct the same canonical metadata before router delivery.

5. Implementation phases

Phase 0 — freeze the experiment contract

- Resolve Gates 0 and 1 before implementation beyond isolated comparison experiments. Resolve Gate 2 before enabling its binding features, Gate 3 before concurrent fixture runs, and Gate 4 before acceptance runs.
- Produce an executable static topology/configuration schema that defines nodes, link profiles and instances/segments, VC attachments and routes, endpoint registrations and roles, peer and listener bindings, forwarders, replicators, queues and bounds, capability/fingerprint declarations, `ProducerKey`s, and a registry of stable validation reason IDs.
- Provide valid minimal and full fixture manifests plus invalid fixtures for conflicting roles or producers, illegal bindings, missing routes/attachments, incompatible capabilities/fingerprints, out-of-range bounds, and unauthorized forwarding/replication.
- Assign temporary literal endpoint IDs, working `ProducerKey`s, and experimental profile labels in generated fixture configuration.
- Write exact experimental semantic schemas, bounds, malformed-input rules, and resource budgets for A–D.
- Create initial positive and negative golden vectors before endpoint behavior is implemented.
- Define capture records, metric names, counter semantics, and deterministic expected deltas needed by every later phase.
- Publish a responsibility/ownership map for codec, endpoint, Service Router, transport, LLL, and Link Entity boundaries. For each boundary it identifies the accepted representation, validation owner and checks, buffer/state ownership transfer and lifetime, and the component that emits the stable rejection/error reason and increments/captures it.

Phase 1 — same-core vertical slice

- Implement fixed storage, Unreliable Datagram pass-through, loopback LLL, router dispatch, and capture.
- Implement Protocol A end to end, then B/C/D.
- Prove request-scoped replies, static autonomous TX, snapshots, event queues, and rejection results.
- Run deterministic malformed-input and congestion tests.

Phase 2 — cross-core local path

- Add the bounded shared-memory LLL and platform memory/coherency adapter.
- Repeat the A–D suite across execution contexts.
- Measure copies, atomics, synchronization, wakeups, latency, and memory cost.
- Restart either side while the other remains active.

Phase 3 — external link and topology

- Add the selected process-isolated virtual datagram/multi-port LLL, or the explicitly approved Gate-0 exception.
- Run F3 and F4, including several Mains and passive same-bus listeners.
- Add the explicit Protocol A replicator and run F5.
- Add the declared same-VC forwarder across two link instances/segments and run F6.

- Verify captures can correlate canonical messages across each configured boundary.

Phase 4 — overload, faults, and recovery

- Execute all negative, congestion, restart, and fault-injection suites.
- Validate persistent counters and DRIP visibility after LLL/router restart.
- Run normal, startup, degraded, diagnostic-burst, and one-node-error-storm load scenarios.
- Produce the resource and capacity report.

Phase 5 — optional link readiness

Add HDLC or CAN only if the owning experimental specification has exact framing/integrity behavior, bounded parser/reassembly state, malformed-input rules, and golden vectors for the subset being tested. Failures found here feed the owning specification; they do not alter A–D semantics.

6. API contracts to prove

The prototype must prove these semantic operations without freezing their spelling or C++ types:

```
registerEndpoint(identity, role, transport, binding, rx_policy, limits)
submit(message, configured_destination) -> bounded TxResult
dispatch(decoded_message, ingress_metadata, opaque_reply_context)
reply(opaque_reply_context, message) -> bounded TxResult
readLatest() -> coherent value plus validity metadata
tryPopEvent() -> event or empty
notifyTxCapacity()
restartLinkOrEntity(reason)
captureAndCount(outcome)
```

Questions the implementation must answer:

- Which checks occur at registration, submit, LLL receive, transport receive, and dispatch?
- When does ownership of a TX buffer transfer, and when may it be reused?
- How is request context retained for deferred replies, invalidated, and rejected after expiry?
- How do separate registrations keep request-scoped responses distinct from autonomous streams/events?
- How do static bindings, and any enabled eligible learned bindings, authorize autonomous downstream messages without treating route metadata as authentication?
- What ordering is guaranteed among multiple local callers?
- Which operations are safe from an ISR, task, host thread, or another core?
- How are capacity notification, shutdown, and restart races bounded?

An ergonomic implementation is evidence for later API design, not authority to standardize the first shape.

7. Validation suites

Functional and topology tests

- A–D happy paths on F1 and F2, then on the external fixture.
- Every enabled peer-binding mode among receive-only, transmit-only, static, request-scoped, and learned; a fixture does not claim a disabled or unsupported mode.
- Receive-only registrations reject every TX and reply attempt with the declared `TxResult`, while transmit-only registrations never acquire, cache, or expose an ingress peer; dedicated captures and counters prove both behaviors and their exact expected deltas.
- Learned binding, when enabled, is tested only for a statically single-peer registration or a named authenticated-origin path, and only against a preconfigured peer allowlist. Route metadata is never accepted as authentication.
- Protocol B preserves its configured roles: the downstream endpoint remains downstream while feedback, current-state streams, and completion events target the configured upstream role.
- Several Main identities on `kMainCompute`.
- Same-bus observation without peer promotion or acknowledgment.
- Explicit multi-bus replication with no fallback/general route.
- Same-VC forwarding through F6 with invariant descriptor, payload, `ProducerKey`/origin context, peer context, and transport state.
- Service instances using the same protocol under different literal endpoint IDs.

Negative protocol and routing tests

- truncated, oversized, and otherwise malformed payloads;
- unsupported `protocol_version` and unknown `message_type`;
- invalid enum/range, nonconforming reserved bits, and wrong profile length;
- wrong namespace, endpoint, transport, role/direction, VC, Index, or `UpstreamHost`;
- either local-host symbol without its required companion, either symbol on external ingress/egress, and any attempted external route for `VirtualCircuit::kLocalHost`;
- alias ID delivered above an LLL, which must be rejected as an implementation/configuration defect;
- unknown endpoint, missing route, incompatible profile, and unexpected ingress;
- forged, expired, reused, or wrong-request reply context;
- for every enabled learned mode, non-allowlisted installation, a non-single-peer unnamed/unauthenticated origin path, route-metadata-only authorization, and bounded-table exhaustion;
- attempted replication of commands, RPC, errors, undeclared Protocol A types, or state from a competing producer;
- listener attempts to reply or affect delivery state.

Parsers are fuzzed within deterministic memory/time bounds. Rejection is counted and captured without recursive diagnostics.

Loss and fault tests

- drop, duplicate, reorder, delay, and burst loss at each datagram boundary;

- producer, consumer, router, LLL, and link restart at every protocol state;
- partial shared-memory record and interrupted TX ownership transition;
- stale snapshot, sequence wrap/discontinuity, clock jump, and producer epoch change;
- external link unavailable, reconnect, and route disabled during traffic;
- replicator input/output failure and restart without loops or stale replay;
- malformed/babbling source and diagnostic collector unavailable.

Link Entity aggregate-status tests

- Every aggregate sample identifies the Link Entity generation and status age so a consumer can distinguish current state from stale state after restart.
- A silent or failed Link Entity becomes degraded/failed within its configured observation bound while healthy sibling Link Entities continue carrying their configured traffic.
- Queue exhaustion and recovery produce the declared state, exact counters, and latched evidence; transient evidence remains observable until the declared acknowledgement/reset policy clears it.
- Status is delivered through a healthy configured service path and does not depend on the failed link or entity being reported.
- An independent debug path, DMA-specific behavior, MCU reset testing, and any platform feature not present in the selected target are conditional capabilities. Their tests run only when enabled in the fixture manifest; omission remains an explicit expected gap and does not fail a platform that declares the feature unsupported.

Congestion and concurrency tests

- reject every submission, reject in bursts, and oscillate capacity;
- verify Protocol A replacement always converges on the newest accepted value;
- fill B/D event rings under each declared overflow policy;
- pause and resume C/D response generation without busy-spin;
- contend all supported multi-caller contexts and verify ordering/ownership rules;
- saturate diagnostic traffic while preserving the configured progress/priority of A/B;
- restart while queues are full and verify disposal versus persistent-state boundaries.

8. Metrics and capture

Before acceptance, the Gate-3/Gate-4 fixture manifest freezes the metric registry and capture schema. Each counter declares width, reset and persistence boundary, saturation behavior, and its precise increment point. Every deterministic test names the expected initial values and exact deltas; probabilistic/fault-timing tests additionally name their seed, clock inputs, tolerance, and permitted outcomes.

Collect per endpoint, route, transport, and link:

- RX, TX, dispatch, success, service-error, malformed, and unexpected-ingress counts;
- accepted, rejected, dropped, replaced/coalesced, suppressed, and retried-submission counts;
- snapshot sequence gaps, stale transitions, and producer restarts;
- event queue occupancy, high-water mark, and overflow;

- request latency, dispatch time, TX queue delay, and scheduling jitter where clocks permit;
- route/replication outcomes and per-output failures;
- link/entity state, restart count/reason/result, and time without progress;
- capture loss and diagnostic-report loss.

Capture records include canonical endpoint/routing metadata, configured `ProducerKey` and provenance, peer context, transport state/profile identity, timestamps with clock-domain/quality, stable outcome/reason ID, and raw-link data where needed for adapter conformance. Capture itself is bounded and reports its own drops according to the same declared counter rules.

9. Resource accounting

For each fixture and service instance, report:

- static and peak RAM by router, routes, packet pool, snapshot, event queue, TX funnel, capture, and diagnostics;
- stack use and any initialization-only allocation;
- flash/code size by common layer, link adapter, codec, and protocol;
- encoded bytes, packet/frame overhead, expected/maximum rate, burst depth, and resulting link load;
- serialization/deserialization time, routing time, end-to-end latency distribution, and throughput;
- copies, atomic operations, lock/critical-section time, cache maintenance, and scheduler wakeups;
- CPU load at idle, normal, worst configured, and fault/diagnostic scenarios.

Queue depths and performance limits are deterministic fixture inputs, not numbers invented by this document. The prototype passes only against budgets and fault tolerances recorded before the corresponding acceptance or stress run. Steady-state MCU paths use no dynamic allocation.

10. Provisional profiles and golden vectors

Every implemented combination has a clearly marked **prototype profile** containing:

- semantic schema revision and protocol version;
- exact A–D message schemas, field bounds, temporary literal EIDs, descriptor packing, byte order, numeric assignments, PDU-length rule, and malformed-input behavior used by the experiment;
- exact virtual-datagram port/segment mapping and transport/LLL configuration;
- experimental profile identity and compatibility-fingerprint inputs;
- message and storage bounds;
- alias, integrity, and reserved-field policy;
- working `ProducerKey` metadata/derivation and provenance rules, explicitly without claiming a frozen wire source field.

Working labels must contain an experimental/prototype marker and must not use `v1` or imply publication. A changed interpretation creates a new working label so captures remain reproducible.

Golden-vector sets contain:

- valid minimum, typical, maximum, boundary, and each-message-type examples;
- byte-exact encoded PDUs plus expected decoded values, canonical metadata, `ProducerKey` /provenance, peer context, and transport state;
- malformed/truncated/oversized cases and expected rejection stage/reason;
- reserved-bit, unknown-version/type, alias-expansion, and routing mismatch cases;
- restart/sequence/correlation transitions that require multiple messages.

At least two independently exercised codecs—initially the target C++ path and a host reference implementation—must agree on positive and negative vectors. FPGA logic may consume the same vectors later. Vectors become candidates for immutable profiles only after the owning consolidated specification resolves its open wire decisions.

11. Acceptance criteria

The initial prototype is accepted when:

- A–D pass their happy, negative, loss, congestion, and restart suites on F1 and F2 and over the selected external link;
- F3 proves several Main identities and normal services over `kMainCompute`;
- F4 proves same-bus listeners without peer promotion, acknowledgment, or loss of configured `ProducerKey` /producer attribution;
- F5 proves only declared Protocol A replication, provenance/conflict policy, rejection of ineligible types and competing producers, and rejection of an attempted general cross-bus ECU route;
- F6 proves same-VC forwarding across two link instances/segments while only hop-local framing changes;
- Protocol B retains downstream/upstream role assignments for request-scoped feedback and autonomous streams/events;
- every enabled binding mode has bounded, tested authorization and lifetime behavior, including observable receive-only and transmit-only criteria; learned binding passes only under the Gate-2 eligibility and allowlist rules;
- aliases are absent above LLL boundaries and all service IDs are canonical literals;
- each TX-capable endpoint has a tested rejection policy and no path busy-spins or grows storage without bound;
- snapshots are coherent, event overflow is exact and observable, and multi-caller ownership/order matches its contract;
- restarts discard uncertain transient state while preserving configured persistent diagnostic evidence;
- nonrecursive diagnostics remain bounded during a babbling-node/error-storm test;
- captures and metrics explain every injected loss, rejection, overflow, and restart without themselves destabilizing traffic;
- Link Entity aggregate status proves generation/age handling, silent/failed detection, healthy sibling continuity, queue-exhaustion behavior, latched evidence, and delivery through a healthy configured path;
- deterministic oracle data records the fixed queue/clock/test values, counter starting points and exact deltas, captures, reason IDs, fault seeds/tolerances, and resource-budget outcomes for every acceptance run;
- both reference codecs pass the same golden vectors;

- measured RAM, flash, CPU, latency, and bandwidth meet the predeclared fixture budgets;
- no steady-state MCU execution requires dynamic allocation;
- unresolved decisions, conditional capability omissions, expected Reliable/CAN/HDLC gaps, and experimental profile labels remain visible in generated reports and documentation.

Passing validates architectural feasibility only. It does not establish safety integrity, security, or production readiness.

12. Explicit exclusions

The initial prototype excludes:

- Reliable transport, reliable RPC, reliable events, and exactly-once claims;
- final E2E protection, cryptographic authentication, confidentiality, or replay protection;
- production Internet/remote access or a general secure gateway;
- dynamic discovery/configuration, runtime route creation, a broker, or general internetwork routing;
- arbitrary ECU-to-ECU unicast across buses;
- transparent conversion between incompatible application schemas;
- bulk object transfer, bootloading, firmware activation, and persistent audit delivery;
- final CAN/HDLC profiles, unless admitted as bounded Phase 5 experiments;
- final service catalog allocations, final code-generation language, or frozen public APIs;
- safety certification or production qualification.

Privileged services, arbitrary packet injection, raw memory access, shells, and update paths are not included.

13. Unresolved-decision gates

Work pauses at a gate only when the choice changes semantics, interoperability, or the validity of a later measurement.

Gate 0 — platforms and external link

Select host/MCU targets, clock sources, scheduler model, and the one external link profile/technology. The default is local host tests plus the process-isolated virtual datagram/multi-port link, instantiated as every configured segment required by F3–F6.

Gate 0 also approves an executable static topology/configuration schema defining nodes, link instances/segments, VC attachments and routes, endpoints and roles, peers, listeners, forwarders, replicators, queues/bounds, capabilities/fingerprints, `ProducerKey`s, and stable validation reason IDs. It includes schema validation and executable invalid fixtures. A non-loopback UDP selection is an exception and is valid only with an isolated non-routed network, exact interface/peer allowlists, host firewall rules, no NAT or forwarding, and the required unauthenticated-carrier warning.

Gate 1 — experimental serialization

Freeze a complete, explicitly experimental prototype profile: exact A–D schemas and bounds, temporary literal EIDs, descriptor packing, byte order, numeric assignments, PDU-length rule, virtual-datagram mapping,

malformed-input behavior, profile identity, compatibility-fingerprint inputs, working `ProducerKey` metadata/derivation and provenance rules, and byte-exact positive and negative vectors. The `ProducerKey` work must not freeze or imply a final wire source field.

Gates 0 and 1 authorize a reproducible experiment, not a stable wire standard. Implementation beyond isolated comparison experiments cannot begin until both gates are resolved; any changed interpretation receives a new experimental profile identity and vectors.

Gate 2 — peer and reply lifetime

Choose request-context retention/expiry and authorization checks for autonomous downstream TX. If learned binding is enabled, define its bounded capacity/eviction/restart policy and restrict installation to a statically single-peer registration or named authenticated-origin path plus a preconfigured peer allowlist. Route metadata is not authentication. Request-scoped responses and autonomous streams/events remain separate endpoint registrations.

Gate 3 — local concurrency

Choose snapshot algorithm, shared-memory ownership, atomics/memory ordering, cache coherency, ISR rules, deterministic queue depths, and multi-caller ordering. Freeze the fixture clock domains, tick/rate/test values, counter widths and reset/saturation/increment semantics, and the concurrency capture fields and expected deltas before running the corresponding fixture.

Gate 4 — congestion and scheduling

Freeze the tested TX-result vocabulary, capacity-notification mechanism, each queue's overflow policy, QoS scheduling mode, metric registry, complete capture schema, remaining counter semantics and exact expected deltas, deterministic fault seeds and tolerances, and predeclared load/resource budgets. Acceptance runs use only versioned manifests containing those values; changing one invalidates the affected run.

Gate 5 — optional HDLC/CAN admission

Admit only the exact experimental subset with complete bounds, integrity coverage, parser/reassembly failure behavior, and vectors. Otherwise defer it without blocking prototype acceptance.

Gate 6 — promotion

After acceptance, decide which APIs, schemas, services, and profiles merit redesign or promotion. Reliable transport begins only with a concrete reliable workload and separate state-machine/resource design.

Until a gate is resolved, the implementation may compare alternatives in isolated tests but must not silently select one as architecture.

14. Local/private security controls

The prototype assumes trusted participants on physically controlled hardware, loopback, shared memory, and process-isolated virtual ports. That assumption is a limitation, not authentication.

- Prefer the process-isolated virtual datagram/multi-port link so the default fixture does not expose a network carrier.

- If Gate 0 admits non-loopback UDP, bind only to the exact approved local interfaces and peers on an isolated non-routed network, enforce host firewall rules, disable NAT and IP forwarding, and do not expose it to the Internet.
- Keep route and endpoint configuration static and allowlisted.
- Disable raw injection and fault controls outside the test harness.
- Separate capture files and debug access from ordinary service authority.
- Rate-limit malformed input and diagnostics even on the private network.
- Record the absence of authentication/confidentiality in every profile and test report, and display the explicit unauthenticated-carrier warning for a UDP exception.

Any trust-boundary crossing requires a separately reviewed gateway/security design; see [rationale](#), [use cases](#), and [risks](#).

Rationale, Use Cases, and Risks

Status: Non-normative

Purpose: Explain design intent, deployment fit, tradeoffs, alternatives, adoption strategy, and risks

Limitation: Examples show architectural fit, not implementation readiness, safety qualification, or wire interoperability

Normative scope and definitions belong to the [architecture overview](#), [core protocol and routing](#), [application protocols and services](#), and [logical links and transports](#).

Why this architecture exists

Embedded projects repeatedly need the same bounded capabilities—commands, state, events, health, configuration, logging, update, and test access—but often redesign them for each CAN bus, UART, FIFO, shared-memory path, or host adapter.

The architecture seeks a reusable middle ground:

- smaller and more analyzable than general internetworking;
- more structured and portable than unrelated raw-link messages;
- usable from local IPC/ICC through bus-connected devices;
- compatible with gateways and outer IP systems without requiring IP on every MCU;
- static enough for route, memory, bandwidth, and failure analysis.

Classical CAN is an important constraint, not the only target. Designing around bounded messages and modest overhead also suits CAN FD, serial links, FIFOs, shared memory, and host datagrams.

Deployment fit

Development board

A PC and one MCU can use the same service model over UART, USB records, Ethernet/UDP, CAN, or a debug-probe bridge. Local host tools can share `VirtualCircuit::kLocalHost`; board traffic uses a configured VC. Firmware identity, health, telemetry, parameters, logs, and test RPC provide an immediate useful subset.

SWD/JTAG remains a separate privileged path. The message fabric does not make processor halt, arbitrary memory access, or flashing safe merely by expressing them as services.

Hardware-in-the-loop system

A HIL rig benefits from explicit producer identity, bounded real-time datagrams, passive observation, capture metadata, simulated peers, and separation between rig-control traffic and device-native traffic. Multiple host processes or Main devices can communicate through local and `kMainCompute` domains.

The orchestrator should not enter every real-time path. Cycle-critical plant simulation belongs on the real-time target and its native I/O. Cross-domain simulation or recording is an explicit proxy/replication function, not accidental general routing.

Industrial robot

The fabric fits bounded controller/drive state, remote I/O, tool services, health, and coordinated status where producers and consumers are configured. Fresh, repeated setpoints are better modeled as time-limited state than as a byte stream.

It is not a replacement for certified safety networks, hardwired safe-torque-off paths, deterministic motion buses, or high-volume vision transport. Safety authority and local enforcement remain explicit and independently justified.

Road vehicle

The model fits compact CAN/CAN-FD services, Main/zone-controller domains, configured listeners on attached links, explicit gateway paths, health and diagnostics, and configured replication of selected state. It supports several Main devices without pretending that every ECU has a globally routable address.

It does not replace established vehicle diagnostics, calibration, safety mechanisms, or high-bandwidth sensor networks merely for architectural uniformity. Telematics and external connectivity must terminate at a policy-enforcing trust boundary; they must not expose raw internal injection.

Satellite

Static configuration, bounded resource use, explicit producer authority, local fault handling, and observable loss/restart behavior fit onboard command, health, and low-rate telemetry domains. The same service model can span redundant onboard links where each path is deliberately configured.

The architecture does not solve disrupted RF communications, mission command authentication, recorder management, or bulk downlink. CCSDS procedures, CFDP-like object transfer, delay-tolerant networking, and mission-specific redundancy remain appropriate outside or above the local message fabric.

Across all five examples, the architecture is most credible for bounded local operational and diagnostic messages. Large objects, media, interactive shells, certified safety channels, and long-delay communications often require specialized adjacent systems.

Central tradeoffs

- **Static configuration over runtime flexibility.** Routes and memberships are analyzable and bounded, but topology changes require configuration and deployment discipline.
- **Communication domains over global addresses.** `VCN + Index` and compact Main identity fit constrained systems, but deliberately prevent arbitrary universal unicast. `UpstreamHost` identifies a configured Main for external/Main relationships; reserved symbolic `UpstreamHost::kLocalHost` represents local IPC/ICC and is distinct from `VirtualCircuit::kLocalHost`. Both numeric encodings remain provisional.
- **Logical consistency over identical wire bytes.** Services can retain semantics across links, while each LLL may use an efficient encoding. This increases the need for precise profiles and conformance tests.
- **Small transport vocabulary over universal behavior.** Each service selects the semantics it needs; integrators must verify compatibility rather than assuming every link and transport combination works.
- **Brokerless publication over dynamic discovery.** Known producers and consumers avoid broker state and discovery traffic, but runtime subscription and ad hoc integration need an explicit external service or proxy.

- **Private-by-default over remote convenience.** Initial exposure is safer and easier to reason about, but remote workflows require intentionally designed gateway infrastructure.
- **Bounded degradation over hidden convenience.** Queue rejection, drops, stale values, and congestion are visible service concerns rather than being hidden behind unbounded buffering.

Why not general routing

General routing would require broader addressing, loop handling, route discovery or convergence, failure policy, more dynamic state, and a larger security surface. Those costs conflict with the target: known nodes, known communication paths, bounded resources, and build-time validation.

Static VCs still permit heterogeneous multi-link paths and configured Main-to-Main traffic. They do not permit arbitrary ECU-to-ECU unicast, particularly across buses. A configured listener on an attached link can consume a visible publication, but listening itself creates no forwarding or fan-out. Transparent forwarding may make the publication visible on another attached link within the same VC; selected cross-VC state replication remains explicit.

Where general routing is actually needed, IP or another established network should provide it outside the embedded fabric. The boundary should be an explicit carrier, proxy, or policy-enforcing gateway.

Why not a mandatory broker

Most target systems know their producers and consumers at integration time. Direct configured delivery avoids a mandatory central service, dynamic subscription state, discovery traffic, and broker failure semantics. It also lets local snapshot storage and bus visibility serve simple one-to-many publication efficiently.

A broker can still be useful on a host, in a lab, or across enterprise systems. It should be an adapter at that boundary, not a prerequisite for an MCU to publish health or consume a command.

Why not a universal byte stream

The primary data model is a discrete bounded message with explicit size, age, producer, and handling policy. A universal reliable stream would add connection state, head-of-line blocking, flow-control coupling, and pressure to buffer data whose age may already make it useless.

Interactive consoles, large logs, firmware images, and payload files remain valid needs. They are better served by a bounded object-transfer service or an established native stream on capable nodes. Their requirements should not determine the cost or failure behavior of every small control endpoint.

Why reliability is not always safer

Delivery reliability and safe application behavior are different properties.

For sampled state or repeated control intent:

- retransmission increases age;
- ordered delivery can block newer data behind an old loss;

- queues can preserve obsolete commands;
- reconnect can replay stale intent;
- acknowledgment proves only the defined peer interaction, not that every observer acted safely.

For these cases, a fresh sequenced value, explicit maximum age, timeout reaction, and current-state feedback can be safer than retransmission. The E2E profile intended to support such checks is still provisional.

Reliability remains valuable for finite transactions and objects: configuration commits, command acceptance, diagnostic operations, and image chunks. However, the Reliable transport is wholly TBD. No reliable state machine, encoding, or interoperability should be inferred from examples in this document.

Adoption and ecosystem strategy

The architecture should earn adoption incrementally rather than require a platform rewrite.

1. Provide immediately useful read-only services and host tools: identity, health, metrics, logging, capture, and configuration fingerprints. Capture is read-only; replay is privileged test-only injection, separately gated and disabled by default.
2. Coexist with existing raw CAN, serial, IPC, and application protocols.
3. Add one service or link at a time, with a clear removal path.
4. Keep core, service libraries, LLLs, platform adapters, and host tools separable.
5. Support bare metal and common RTOS environments without imposing a task model, allocator, build system, or application framework.
6. Treat Linux tooling, simulation, capture, and Python integration as product capabilities rather than afterthoughts.
7. Demonstrate acceptable CAN efficiency and bounded MCU cost with measured prototypes.
8. Publish exact profiles and conformance vectors before claiming cross-implementation interoperability.

The strongest adoption wedge is likely a useful service/tool pair, not the abstract protocol alone. A health or boot service that works across several targets and links is more persuasive than a broad architecture without production evidence.

Principal risks

Architecture and implementation

- **Architecture before proof.** Interfaces may appear clean until real CAN, serial, multicore, congestion, and restart behavior is measured.
- **Configuration failure.** Static routing trades protocol dynamics for code-generation, review, rollout, and fingerprint consistency risks.
- **Leaky link differences.** Message limits, arbitration, ordering, integrity, and medium access cannot be erased by a common API.
- **Resource surprises.** Queue state, copies, transport state, serialization, and diagnostics may exceed MCU budgets.
- **False maturity.** A coherent descriptor or draft packet layout is not an interoperable profile.

- **Producer ambiguity.** Incorrect simulation, failover, or replication can create competing sources for control-relevant state.
- **Overload cascades.** Diagnostics, retries, or forwarding can amplify a failing node unless bounded and rate-limited.

The [prototype and validation](#) plan should test these risks with heterogeneous links, congestion, stale configuration, restart, malformed input, and measured CPU/RAM/wire costs.

Security and debug

The base fabric does not authenticate peers, encrypt traffic, authorize operations, or provide cryptographic replay protection. Private topology reduces exposure; it does not make a compromised node trustworthy.

High-risk capabilities include:

- arbitrary memory or MMIO access;
- unrestricted shell or RPC execution;
- replay or raw CAN/link injection;
- fault injection and control overrides;
- firmware update and boot control;
- JTAG/SWD, calibration, trace, and processor halt;
- broad forwarding from an IP-connected host.

The initial prototype excludes shell, memory/MMIO access, update, fault control, replay, and raw injection. Experimental versions require a separate development build, remain separately gated and disabled by default, and cannot be enabled solely by runtime network configuration. Capture remains read-only. Read-only and write/control authority must be distinct. Diagnostic traffic must be bounded so a faulty node cannot destabilize healthy control traffic.

A future remote gateway needs a reviewed threat model, strong peer and message authentication, authorization by service/operation/domain, replay protection, key lifecycle, rate limiting, audit, and safe session/reset behavior. Tunneling through TLS or a VPN alone does not establish correct endpoint authorization.

Alternatives and when to use them

- **Raw CAN plus a DBC or project-specific messages** — appropriate for a small fixed system where minimal overhead and existing tooling outweigh cross-link reuse.
- **CANopen, J1939, UDS/DoIP, XCP, AUTOSAR communication, or another domain stack** — preferable when its standardized services, compliance, ecosystem, or vehicle integration already match the requirement.
- **UDP/IP or TCP/IP** — preferable when routability, established security tooling, sockets, and host interoperability matter more than minimal MCU overhead.
- **DDS, SOME/IP, Zenoh, MQTT, or another broker/discovery ecosystem** — preferable when dynamic discovery, rich QoS, enterprise integration, or large-node environments justify the footprint and operational model.

- **Native RTOS IPC or shared-memory APIs** — preferable for tightly local interactions that do not benefit from service portability or network placement.
- **Dedicated deterministic or certified safety networks** — required when scheduling, certification, redundancy, or safety evidence exceeds this fabric's scope.
- **CCSDS, CFDP, or disruption-tolerant networking** — preferable for space links, long delay, scheduled contacts, and resilient object delivery.

These are not mutually exclusive choices. A deployment can use the embedded fabric for bounded local services and established systems for diagnostics, bulk transfer, safety, media, or external networking.

Future research

- Final canonical semantics, numeric assignments, immutable named profiles, and golden vectors.
- Independent implementations and differential/conformance testing.
- Complete E2E sequencing, identity, freshness, reset, CRC/MAC, and failure behavior.
- Reliable bounded-message requirements and state machine, derived from concrete transactions and object transfer.
- Experimental CAN and HDLC profile completion, malformed-input handling, and measured efficiency.
- Generated VC/route configuration with loop, reachability, ingress, capacity, and compatibility validation.
- Bounded scheduling, backpressure, reservation, overload, and diagnostic-storm analysis.
- Recovery/status semantics for link loss, endpoint restart, stale state, and configuration mismatch.
- Local IPC/ICC copy, cache-coherency, synchronization, queue, and placement costs.
- Static failover and redundant-path behavior without introducing general dynamic routing.
- Optional end-to-end authentication and a secure, least-privilege remote gateway architecture.
- Evidence needed for safety-relevant, industrial, vehicle, and space use.

Research outcomes become authoritative only when incorporated into their owning consolidated specification and validated. This document does not assign unresolved numbers, freeze wire layouts, or assert interoperability.

Related documents

- [Document index and source traceability](#)
- [Architecture overview](#)
- [Core protocol and routing](#)
- [Application protocols and services](#)
- [Logical links and transports](#)
- [Link Entity runtime and status](#)
- [HDLC logical-link profile](#)
- [CAN PDU adapter specification](#)
- [Prototype and validation](#)